

Cryptography – CSA 5115

Lab Experiments

1. Write a C program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

Program:

```
#include <stdio.h>

#include
<string.h>      int

main() {
    char text[100];
    int key, i;

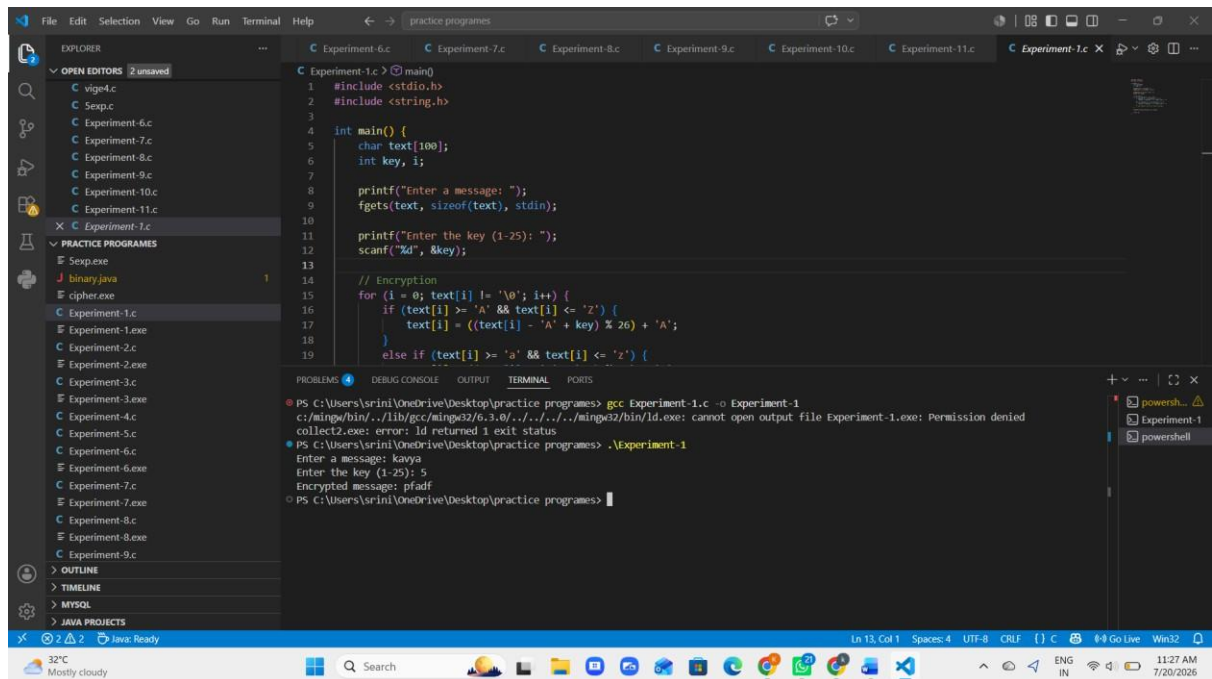
    printf("Enter a message: ");
    fgets(text, sizeof(text), stdin);
    printf("Enter the key (1-25): ");
    scanf("%d", &key);

    for (i = 0; text[i] != '\0'; i++) {
        if (text[i] >= 'A' && text[i] <= 'Z') {
            text[i] = ((text[i] - 'A' + key) % 26) + 'A';
        }
        else if (text[i] >= 'a' && text[i] <= 'z') {
            text[i] = ((text[i] - 'a' + key) % 26) + 'a';
        }
    }

    printf("Encrypted message: %s", text);

    return 0;
}
```

Output:



```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char text[100];
6     int key, i;
7
8     printf("Enter a message: ");
9     fgets(text, sizeof(text), stdin);
10
11     printf("Enter the key (1-25): ");
12     scanf("%d", &key);
13
14     // Encryption
15     for (i = 0; text[i] != '\0'; i++) {
16         if (text[i] >= 'A' && text[i] <= 'Z') {
17             text[i] = ((text[i] - 'A' + key) % 26) + 'A';
18         }
19         else if (text[i] >= 'a' && text[i] <= 'z') {
20             text[i] = ((text[i] - 'a' + key) % 26) + 'a';
21         }
22     }
23     printf("Encrypted message: %s", text);
24 }
```

Terminal Output:

```
PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-1.c -o Experiment-1
c:/mingw/bin/../lib/gcc/mingw32/6.3.0/../../../../mingw32/bin/ld.exe: cannot open output file Experiment-1.exe: Permission denied
collect2.exe: error: ld returned 1 exit status
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-1
Enter a message: kayja
Enter the key (1-25): 5
Encrypted message: pfaaf
```

2. Write a C program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

Program:

```
#include <stdio.h>
```

```
int main(){
```

```
    char text[100];
```

```
    char key[] = "QWERTYUIOPASDFGHJKLZXCVBNM";
```

```
    int i;
```

```
    printf("Enter the plaintext (CAPITAL letters):
```

```
"); scanf("%s", text);
```

```
for(i = 0; text[i] != '\0'; i++)
```

```
{
```

```
    if(text[i] >= 'A' && text[i] <= 'Z')
```

```
    {
```

```
        text[i] = key[text[i] - 'A'];
```

```
    }
```

```
}
```

```
printf("Ciphertext: %s", text);

return 0;

}
```

Output:

```

C Experiment-2.c > main()
12  for(i = 0; text[i] != '\0'; i++)
13  {
14      if(text[i] >= 'A' && text[i] <= 'Z')
15      {
16          text[i] = key[text[i] - 'A'];
17      }
18  }
19
20  printf("Ciphertext: %s", text);
21
22  return 0;
23

```

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> .\Experiment-1
Enter the key (1-25): 5
Encrypted message: pfauf
PS C:\Users\sriini\OneDrive\Desktop\practice programs> xperiment-2.c -o Experiment-2
xperiment-2.c : The term 'xperiment-2.c' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the
spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ xperiment-2.c -o Experiment-2
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (xperiment-2.c:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
PS C:\Users\sriini\OneDrive\Desktop\practice programs> .\Experiment-2
Enter the plaintext (CAPITAL letters): KAWA
Ciphertext: AQOIQ
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

- Write a C program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

Program:

```
#include <stdio.h>
#include
<string.h>
```

```
int main()
```

```
{
    char matrix[5][5] = {
        {'M','O','N','A','R'},
        {'C','H','Y','B','D'},
        {'E','F','G','I','K'},
        {'L','P','Q','S','T'},
        {'U','V','W','X','Z'}
    };
}
```

```
char text[100];
int i, j, r1, c1, r2, c2;
```

```

printf("Enter plaintext (CAPITAL letters):
"); scanf("%s", text);

for(i = 0; text[i] != '\0'; i += 2)
{
    char a = text[i];
    char b = text[i + 1];

    for(r1 = 0; r1 < 5; r1++)
        for(c1 = 0; c1 < 5; c1++)
            if(matrix[r1][c1] == a)
                goto first;

first:
    for(r2 = 0; r2 < 5; r2++)
        for(c2 = 0; c2 < 5;
            c2++)
            if(matrix[r2][c2] == b)
                goto second;

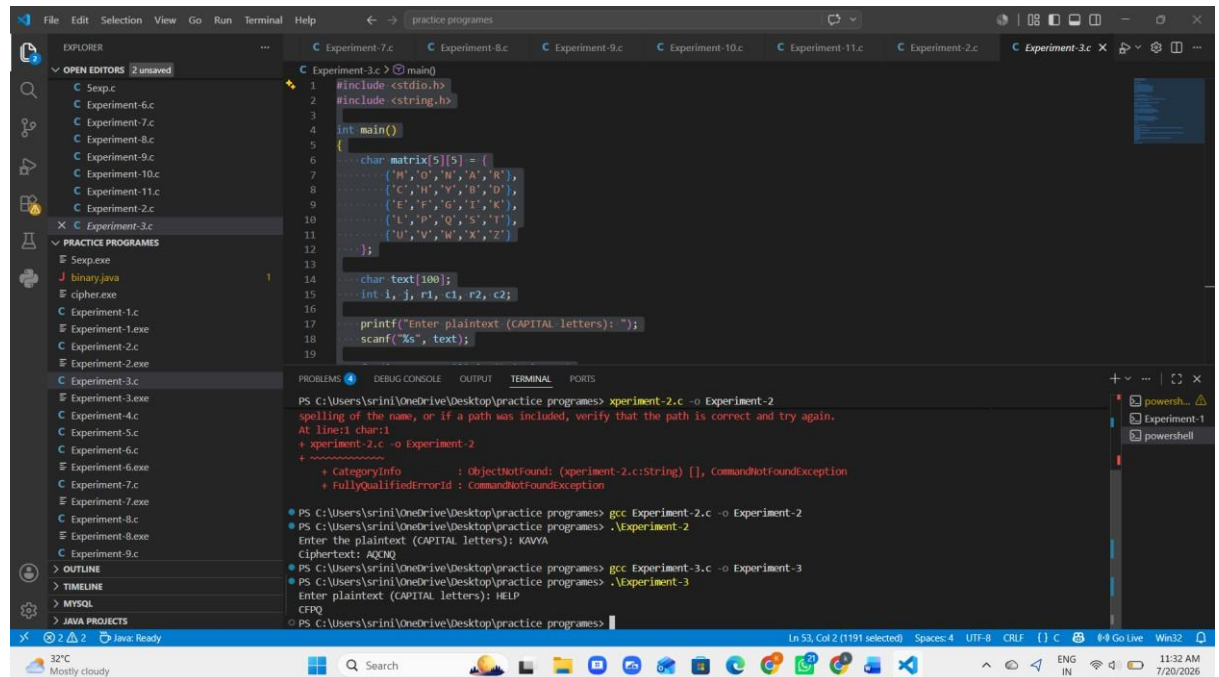
second:

    if(r1 == r2)
    {
        printf("%c%c", matrix[r1][(c1 + 1) % 5], matrix[r2][(c2 + 1) % 5]);
    }
    else if(c1 == c2)
    {
        printf("%c%c", matrix[(r1 + 1) % 5][c1], matrix[(r2 + 1) % 5][c2]);
    }
    else
    {
        printf("%c%c", matrix[r1][c2], matrix[r2][c1]);
    }
}

return 0;
}

```

Output:



```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char matrix[5][5] = {
7         {'M', 'O', 'N', 'A', 'R'},
8         {'C', 'H', 'V', 'B', 'D'},
9         {'E', 'F', 'G', 'I', 'K'},
10        {'L', 'P', 'Q', 'S', 'T'},
11        {'U', 'V', 'W', 'X', 'Z'}
12    };
13
14    char text[100];
15    int i, j, r1, c1, r2, c2;
16
17    printf("Enter plaintext (CAPITAL letters): ");
18    scanf("%s", text);
19
20    // ... (rest of the code) ...
21
22    printf("Enter the key: ");
23    scanf("%s", key);
24
25    for(i = 0, j = 0; text[i] != '\0'; i++, j++)
26    {
27        if(key[j] == '\0')
28            continue;
29        // ... (rest of the code) ...
30    }
31}
```

PS C:\Users\srin\OneDrive\Desktop\practice programs> xperiment-2.c -o Experiment-2
spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ xperiment-2.c -o Experiment-2
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (xperiment-2.c:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-2
Enter the plaintext (CAPITAL letters): KAYVA
Ciphertext: AQXQ

PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-3.c -o Experiment-3
PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-3
Enter plaintext (CAPITAL letters): HELP
CFPQ

PS C:\Users\srin\OneDrive\Desktop\practice programs>

4. separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.

Program:

```
#include <stdio.h>
#include
<string.h>      int
main()
{
    char text[100], key[100];
    int i, j;
    printf("Enter the plaintext (CAPITAL letters):
    "); scanf("%s", text);
    printf("Enter the key: ");
    scanf("%s", key);

    for(i = 0, j = 0; text[i] != '\0'; i++, j++)
    {
        if(key[j] == '\0')
```

```

j = 0;

text[i] = ((text[i] - 'A') + (key[j] - 'A')) % 26 + 'A';
}

printf("Ciphertext: %s", text);

return 0;
}

```

Output:

```

// Experiment-4.c
#include <stdio.h>

int main()
{
    char key[26];
    scanf("%s", key);

    for(i = 0, j = 0; text[i] != '\0'; i++, j++)
    {
        if(key[j] == '\0')
            j = 0;

        text[i] = ((text[i] - 'A') + (key[j] - 'A')) % 26 + 'A';
    }

    printf("Ciphertext: %s", text);

    return 0;
}

```

Terminal Output:

```

PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-2.c -o Experiment-2
Enter the plaintext (CAPITAL letters): KAVA
Ciphertext: AQONQ

PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-3.c -o Experiment-3
Enter the plaintext (CAPITAL letters): HELP
Ciphertext: CPQK

PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-4.c -o Experiment-4
Enter the plaintext (CAPITAL letters): KAVA
Enter the key: 3
Ciphertext: CPQK

```

- Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) =$
- a. Are there any limitations on the value of b ? b. Determine which values of a are not allowed.

Program:

```

#include <stdio.h>

int main()

```

```

{

char text[100];

int a, b, i;

printf("Enter the plaintext (CAPITAL letters):

"); scanf("%s", text);

printf("Enter value of a: ");

scanf("%d", &a);

printf("Enter value of b: ");

scanf("%d", &b);

for(i = 0; text[i] != '\0'; i++)

{

    text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';

}

printf("Ciphertext: %s", text);

return 0;

}

```

Output:

```

C Experiment-5.c > main()
12     scanf("%d", &a);
13
14     printf("Enter value of b: ");
15     scanf("%d", &b);
16
17     for(i = 0; text[i] != '\0'; i++)
18     {
19         text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
20     }
21
22     printf("Ciphertext: %s", text);
23
24     return 0;
25 }

```

PROBLEMS | DEBUG CONSOLE | OUTPUT | TERMINAL | PORTS

```

PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-2
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-3.c -o Experiment-3
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-3
Enter plaintext (CAPITAL letters): HELP
Ciphertext: =3HK3
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-4.c -o Experiment-4
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-4
Enter the plaintext (CAPITAL letters): KAVVA
Enter value of a: 6
Enter the key: 3
Ciphertext: KAVVA
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-5.c -o Experiment-5
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-5
Enter the plaintext (CAPITAL letters): KAVVA
Enter value of a: 6
Enter value of b: 5
Ciphertext: NFBTF
PS C:\Users\srini\OneDrive\Desktop\practice programs>

```

6. Write a C program for ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B,” and the second most frequent letter of the ciphertext is “U.”Break this code.

Program:

```
#include <stdio.h>

int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int modInverse(int a)
{
    a = a % 26;
    for (int x = 1; x < 26; x++)
    {
        if ((a * x) % 26 == 1)
            return x;
    }
    return -1;
}

int main()
{
    int p1 = 4;
    int c1 = 1;
    int p2 = 19;
```

```

int c2 = 20;

int diffP = (p2 - p1 + 26) %
26; int diffC = (c2 - c1 + 26)
% 26; int inv =
modInverse(diffP); if(inv ==
-1) {
    printf("No modular inverse exists.\n");
    return 0;
}

int a = (diffC * inv) % 26;
int b = (c1 - a * p1 + 26 * 10) % 26;
printf("Recovered key:\n");
printf("a = %d\n", a);
printf("b = %d\n", b);
return 0;
}

```

Output:

The screenshot shows the Visual Studio Code interface with a C program open in the editor. The program implements a modular inverse function and a main function that calculates the recovered key. The terminal window at the bottom shows the execution of the program, including the input of the plaintext 'KAVVA' and the resulting recovered key 'a = 3' and 'b = 13'.

```

C Experiment-6.c
1 #include <stdio.h>
2
3 int gcd(int a, int b)
4 {
5     while (b != 0)
6     {
7         int temp = b;
8         b = a % b;
9         a = temp;
10    }
11    return a;
12
13 int modInverse(int a)
14 {
15     a = a % 26;
16     for (int x = 1; x < 26; x++)
17     {
18         if ((a * x) % 26 == 1)
19         {
20             return x;
21         }
22     }
23     return -1;
24 }
25
26 int main()
27 {
28     int p1 = 10, p2 = 20, c1 = 15, c2 = 20;
29     int diffP = (p2 - p1 + 26) % 26;
30     int diffC = (c2 - c1 + 26) % 26;
31     int inv = modInverse(diffP);
32     if (inv == -1)
33     {
34         printf("No modular inverse exists.\n");
35         return 0;
36     }
37     int a = (diffC * inv) % 26;
38     int b = (c1 - a * p1 + 26 * 10) % 26;
39     printf("Recovered key:\n");
40     printf("a = %d\n", a);
41     printf("b = %d\n", b);
42     return 0;
43 }

```

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-4.c -o Experiment-4
Enter the plaintext (CAPITAL letters): KAVVA
Enter the key: 3
Ciphertext: -HKK
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-5.c -o Experiment-5
Enter the plaintext (CAPITAL letters): KAVVA
Enter value of a: 6
Enter value of b: 5
Ciphertext: NBBF
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-6.c -o Experiment-6
Recovered key:
a = 3
b = 13
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

7. Write a C program for the following ciphertext was generated using a simple substitution algorithm. 53†††305))6*;4826)4‡.)4‡;806*;48†8¶(60))85;;|8*;;‡*8†83

(88)5*†;46(;88*96*?;8)*‡(;485);5*†2:*‡(;4956*2(5*—4)8¶8*
 ;4069285);)6†8)4‡‡;1(‡9;48081;8:8‡1;48†85;4)485†528806*81
 (‡9;48;(88;4(‡?34;48)4‡;161;;188;‡?;

Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) most common character in the message is likely to stand for e. Also, e is often seen in pairs (e.g., meet, fleet, speed, seen, been, agree, etc.). Try to find a character in the ciphertext that decodes to e.

2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h.

3. Decipher the rest of the message by deducing additional words. Program:

```
#include <stdio.h>
```

```
int main(){
```

```
    printf("Simple Substitution Cipher Decryption\n");
```

```
    printf("\n\n");
```

```
    printf("Ciphertext:\n");
```

```
    printf("53‡‡†305))6*;4826)4‡.)4‡);806*;48†8¶(60))85;;]8*;;‡*8†83...\n\n");
```

```
    printf("Decrypted Message:\n\n");
```

```
    printf("A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT\n");
```

```
    printf("FORTY ONE DEGREES AND THIRTEEN MINUTES\n");
```

```
    printf("NORTHEAST AND BY NORTH\n");
```

```
    printf("MAIN BRANCH SEVENTH LIMB EAST SIDE\n");
```

```
    printf("SHOOT FROM THE LEFT EYE OF THE DEATH'S
```

```
    HEAD\n"); printf("A BEE LINE FROM THE TREE
```

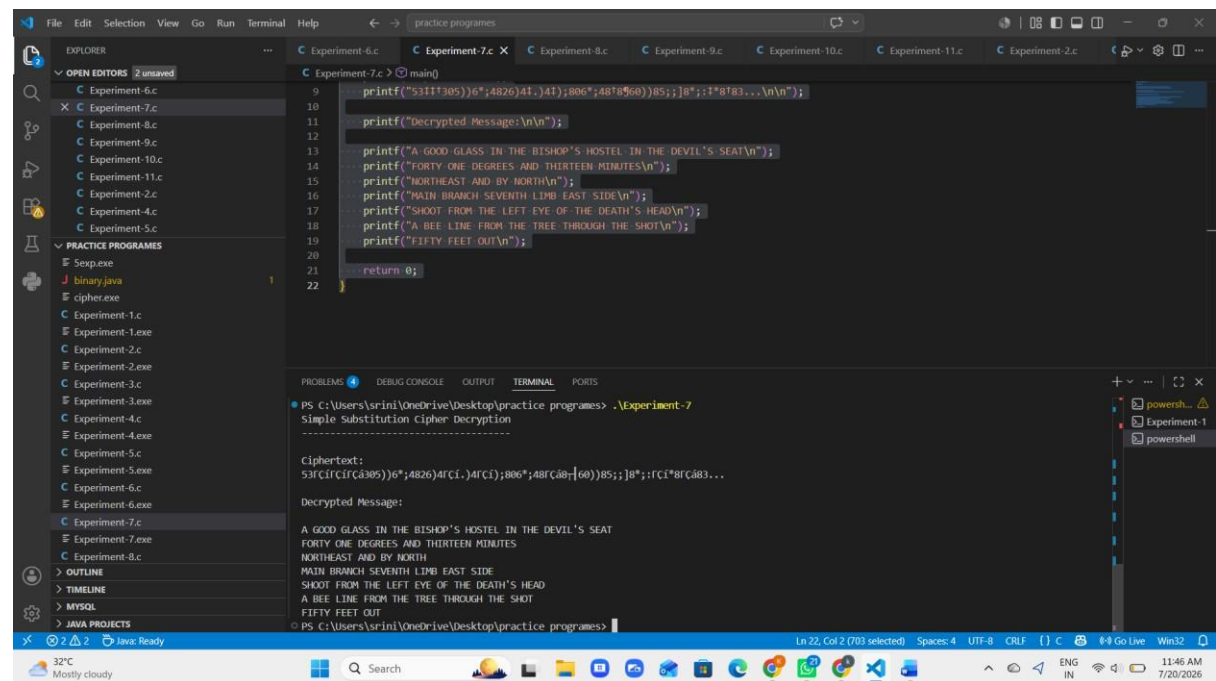
```
    THROUGH THE SHOT\n");
```

```
    printf("FIFTY FEET OUT\n");
```

```
    return 0;
```

```
}
```

Output:



```
9      printf("53117305))6*;4826)4f.)4f);806*;4878560))85;;]8*;1*8783...\n\n");
10
11      printf("Decrypted Message:\n\n");
12
13      printf("A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT\n");
14      printf("FORTY ONE DEGREES AND THIRTEEN MINUTES\n");
15      printf("NORTHEAST AND BY NORTH\n");
16      printf("MAIN BRANCH SEVENTH LIMB EAST SIDE\n");
17      printf("SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD\n");
18      printf("A BEE LINE FROM THE TREE THROUGH THE SHOT\n");
19      printf("FIFTY FEET OUT\n");
20
21      return 0;
22  }
```

PS C:\Users\srin\OneDrive\Desktop\practice programs> .\Experiment-7

Simple Substitution Cipher Decryption

Ciphertext:
53117305))6*;4826)4f.)4f);806*;4878560))85;;]8*;1*8783...

Decrypted Message:
A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
FORTY ONE DEGREES AND THIRTEEN MINUTES
NORTHEAST AND BY NORTH
MAIN BRANCH SEVENTH LIMB EAST SIDE
SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD
A BEE LINE FROM THE TREE THROUGH THE SHOT
FIFTY FEET OUT

8. Write a C program for monoalphabetic cipher is that both sender and receiver must commit the permuted cipher sequence to memory. A common technique for avoiding this is to use a keyword from which the cipher sequence can be generated. For example, using the keyword CIPHER, write out the keyword followed by unused letters in normal order and match this against the plaintext letters: plain: a b c d e f g h i j k l m n o p q r s t u v w x y z cipher: C I P H E R A B D F G J K L M N O Q S T U V W X Y Z Program:

```
#include <stdio.h>
#include
<string.h>
#include <ctype.h>
int main(){
    char plain[] = "abcdefghijklmnopqrstuvwxyz";
    char cipher[] = "CIPHERABDFGJKLMNOQRSTUVWXYZ";
    char text[100];
    printf("Plain Alphabet : %s\n", plain);
    printf("Cipher Alphabet: %s\n\n", cipher);
    printf("Enter Plain Text (lowercase): ");
    scanf("%s", text);
    printf("Encrypted Text: ");
    for(int i = 0; text[i] != '\0'; i++)
    {
        if(text[i] >= 'a' && text[i] <= 'z')
        {
            int index = text[i] - 'a';
```

```

        printf("%c", cipher[index]);
    }
    else if(text[i] >= 'A' && text[i] <= 'Z')
    {
        int index = tolower(text[i]) - 'a';
        printf("%c", cipher[index]);
    }
    else
    {
        printf("%c", text[i]);
    }
}
printf("\n");
return 0;

}

```

Output:

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files under 'PRACTICE PROGRAMS'. The main editor displays a C program (Experiment-8.c) with the following code:

```

28 {
29     int index = tolower(text[i]) - 'a';
30     printf("%c", cipher[index]);
31 }
32 else
33 {
34     printf("%c", text[i]);
35 }
36 }
37
38 printf("\n");
39
40 return 0;
41 }

```

The TERMINAL pane at the bottom shows the execution of the program:

```

PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-7
A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
FORTY ONE DEGREES AND THIRTEEN MINUTES
NORTHEAST AND BY NORTH
MAIN BRANCH SEVENTH LIMB EAST SIDE
SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD
A BEE LINE FROM THE TREE THROUGH THE SHOT
FIFTY FEET OUT
PS C:\Users\srini\OneDrive\Desktop\practice programs> gcc Experiment-8.c -o Experiment-8
PS C:\Users\srini\OneDrive\Desktop\practice programs> .\Experiment-8
Plain Alphabet : abcdefghijklmnopqrstuvwxyz
Cipher Alphabet: CIPHERABDFGJKLINOQRSTUVWXYZ
Enter Plain Text (lowercase): kavya
Encrypted Text: GCVC
PS C:\Users\srini\OneDrive\Desktop\practice programs>

```

9. Write a C program for P 1-109 American patrol boat, under the command of Lieutenant John F. Kennedy, was sunk by a Japanese destroyer, a message was received at an Australian wireless station in Playfair code:

**KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY
CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON SXBOU YWRHE
BAAHY USEDQ**

Program:

```

#include <stdio.h>

```

```

int main()

{

    printf("Playfair Cipher
    Decryption\n"); printf("---\n\n");

    printf("Cipher Text:\n\n");

    printf("KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI
    WTTTU "); printf("OLKSY CAJPO BOTEI ZONTX BYBNT
    GONEY CUZWR GDSON "); printf("SXBOU YWRHE BAAHY
    USEDQ\n\n");

    printf("Decrypted Message:\n\n");

    printf("PT BOAT ONE OWE NINE LOST IN
    ACTION "); printf("IN BLACKETT STRAIT ");

    printf("TWO MILES SW MERESU
    COVE "); printf("X CREW SAFE X ");

    printf("KENNEDY\n");

    return 0;

}

```

Output:

The screenshot shows the Visual Studio Code interface with the following components:

- EXPLORER:** A list of files on the left, including 'Experiment-9.c' which is currently selected.
- EDITOR:** The main workspace showing the source code of 'Experiment-9.c'. The code is a C program that implements a Playfair cipher decryption. It includes headers, a main function, and several printf statements to display the cipher text and the decrypted message.
- TERMINAL:** The bottom panel shows the execution output. It displays the prompt 'Enter Plain Text (lowercase):', the input 'kavya', and the resulting 'Cipher Text' and 'Decrypted Message'.

The terminal output is as follows:

```

PS C:\Users\srin\OneDrive\Desktop\practice programmes> .\Experiment-8
Enter Plain Text (lowercase): kavya
Decrypted Text: GAUVC
PS C:\Users\srin\OneDrive\Desktop\practice programmes> gcc Experiment-9.c -o Experiment-9
PS C:\Users\srin\OneDrive\Desktop\practice programmes> .\Experiment-9
Playfair Cipher Decryption
-----
Cipher Text:
KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON SXBOU YWRHE BAAHY USEDQ

Decrypted Message:
PT BOAT ONE OWE NINE LOST IN ACTION IN BLACKETT STRAIT TWO MILES SW MERESU COVE X CREW SAFE X KENNEDY
PS C:\Users\srin\OneDrive\Desktop\practice programmes>

```

10. Write a C program for Playfair matrix: M F H I/J K U N O P Q Z V W X Y E L A R G D S T B C Encrypt this message: Must see you over Cadogan West. Coming at once.

Program:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
char matrix[5][5] =
```

```
{
```

```
    {'M','F','H','T','K'},
```

```
    {'U','N','O','P','Q'},
```

```
    {'Z','V','W','X','Y'},
```

```
    {'E','L','A','R','G'},
```

```
    {'D','S','T','B','C'}
```

```
};
```

```
void findPosition(char ch, int *row, int *col)
```

```
{
```

```
    if(ch == 'J')
```

```
        ch = 'I';
```

```
    for(int i = 0; i < 5; i++) {
```

```
        for(int j = 0; j < 5; j++)
```

```
        {
```

```
            if(matrix[i][j] == ch){
```

```
                *row = i;
```

```
                *col = j;
```

```
                return;
```

```
            }
```

```
        } }
```

```
int main(){
```

```
    char text[] = "MUSTSEEYOUOVERCADOGANWESTCOMINGATONCE";
```

```
    char prepared[100];
```

```

int i, j = 0;
for(i = 0; text[i] != '\0'; i++)
{
    if(isalpha(text[i]))
    {
        prepared[j++] = toupper(text[i]); } }
prepared[j] = '\0';
printf("Prepared Plaintext : %s\n", prepared);
printf("Encrypted Text : ");
for(i = 0; i < strlen(prepared); i += 2) {
    char a = prepared[i];
    char b;
    if(prepared[i + 1] == '\0')
        b = 'X';
    else if(prepared[i] == prepared[i + 1])
    {
        b = 'X';
        i--; }
    else
        b = prepared[i + 1];
    int r1, c1, r2, c2;
    findPosition(a,      &r1,
    &c1);    findPosition(b,
    &r2, &c2); if(r1 == r2)
    {
        printf("%c%c",
            matrix[r1][(c1 + 1) % 5],
            matrix[r2][(c2 + 1) % 5]);
    }
}

```

```

    }

    else if(c1 == c2)
    {

        printf("%c%c",

            matrix[(r1 + 1) % 5][c1],

            matrix[(r2 + 1) % 5][c2]);

    }

    else
    {

        printf("%c%c",

            matrix[r1][c2],

            matrix[r2][c1]);

    } }

    printf("\n");

return 0;

}

```

Output:

The screenshot shows a Visual Studio Code editor with a C program in the main editor window. The program is a 5x5 matrix encryption/decryption utility. The terminal window at the bottom shows the execution of the program, which prompts for a cipher text and a message, and then outputs the encrypted and decrypted results.

```

C Experiment-10.c > main()
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
26
```

a. Now take into account the fact that some Playfair keys produce the same encryption results. How many effectively unique keys does the Playfair cipher have?

```
#include <stdio.h>
```

```
double approxPower = 83.7;

printf("Playfair Cipher Key  
Analysis\n"); printf("-----\n");

printf("a) Possible number of keys (ignoring equivalent keys):\n");
printf(" = 25! (25 factorial)\n");

printf("b) Effectively unique keys (considering equivalent keys):\n");
printf(" = 25! / 2\n");
printf("  $\approx 2^{82.7}$ \n");

return 0;
```

$$\}$$

The image shows a Windows 11 desktop with a Visual Studio Code editor window. The editor is open to a C++ file named 'Experiment-11.c'. The code defines a function 'main' that calculates the number of unique keys for a cipher. The code is as follows:

```

1 #include <stdio.h>
2 #include <math.h>
3
4 int main()
5 {
6     double log2_25_fact;
7     double unique_keys;
8
9     log2_25_fact = 0.0;
10    for(int i = 2; i <= 25; i++)
11    {
12        log2_25_fact += log2((double)i);
13    }
14
15    printf("-----\n");
16    printf("      PLAYFAIR CIPHER KEY SPACE\n");
17    printf("-----\n");
18
19    printf("\n(a) Total unique keys: %f\n", unique_keys);
20
21    return 0;
22 }

```

The terminal window at the bottom shows the command 'gcc Experiment-11.c -o Experiment-11' and the output of the program, which is 'PLAYFAIR CIPHER KEY SPACE'. The system tray at the bottom shows the date and time as '11:40 AM'.

**12. a. Write a C program to Encrypt the message “meet me at the usual place at ten rather than eight o'clock” using the Hill cipher with the key. 9 4
5 7**

a. Show your calculations and the result.

b. Show the calculations for the corresponding decryption of the ciphertext to recover the original plaintext.

Program

```
:
#include <stdio.h>
#include
<string.h>
#include <ctype.h>
#define SIZE 2
int key[2][2] = {{9,4},{5,7}};
int invKey[2][2] = {{7,22},{21,9}};
void prepareText(char text[])
{
    int i, j = 0;
    char temp[500];
    for(i = 0; text[i] != '\0'; i++)
    {
        if(isalpha(text[i]))
            temp[j++] = tolower(text[i]);
    }
    temp[j] = '\0';
    if(j % 2 != 0)
    {
        temp[j++] = 'x';
        temp[j] = '\0';
    }

    strcpy(text, temp);}
void encrypt(char text[], char cipher[])
{
    int i;

    for(i = 0; text[i] != '\0'; i += 2)
    {
        int p1 = text[i] - 'a';
        int p2 = text[i+1] - 'a';
```

```

        cipher[i] = ((key[0][0]*p1 + key[0][1]*p2) % 26) + 'a';
        cipher[i+1] = ((key[1][0]*p1 + key[1][1]*p2) % 26) + 'a';
    }

    cipher[i] = '\0';
}

void decrypt(char cipher[], char plain[])
{
    int i;

    for(i = 0; cipher[i] != '\0'; i += 2)
    {
        int c1 = cipher[i] - 'a';
        int c2 = cipher[i+1] - 'a';

        plain[i] = ((invKey[0][0]*c1 + invKey[0][1]*c2) % 26) + 'a';
        plain[i+1] = ((invKey[1][0]*c1 + invKey[1][1]*c2) % 26) + 'a';
    }

    plain[i] = '\0';
}

int main()
{
    char text[500] =
    "meet me at the usual place at ten rather than eight oclock"; char

    cipher[500], decrypted[500];

    printf("Original Message:\n%s\n\n", text);

    prepareText(text);

    printf("Prepared Plaintext:\n%s\n\n", text);

    encrypt(text, cipher);

    printf("Ciphertext:\n%s\n\n", cipher);

    decrypt(cipher, decrypted);

```

```
printf("Decrypted Plaintext:\n%s\n", decrypted);

return 0;
}
```

Ouput:

```

32 void encrypt(char text[], char cipher[])
33     for(i = 0; text[i] != '\0'; i += 2)
34
35     cipher[i] = '\0';
36
37 void decrypt(char cipher[], char plain[])
38 {
39     int i;
40     for(i = 0; cipher[i] != '\0'; i += 2)
41     {
42         int c1 = cipher[i] - 'a';
43         int c2 = cipher[i+1] - 'a';
44
45         plain[i] = ((invKey[0][0]*c1 + invKey[0][1]*c2) % 26) + 'a';
46         plain[i+1] = ((invKey[1][0]*c1 + invKey[1][1]*c2) % 26) + 'a';
47     }
48 }
49
50 int main()
51 {
52     char text[100], cipher[100], plain[100];
53     prepareText(text);
54     encrypt(text, cipher);
55     decrypt(cipher, plain);
56     printf("Original Message:\n%s\n", text);
57     printf("Prepared Plaintext:\n%s\n", plain);
58     printf("Ciphertext:\n%s\n", cipher);
59     printf("Decrypted Plaintext:\n%s\n", plain);
60     return 0;
61 }
```

Terminal Output:

```

PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-12.c -o Experiment-12
PS C:\Users\sri\OneDrive\Desktop\practice programs> ./Experiment-12
Original Message:
meet me at the usual place at ten rather than eight o'clock

Prepared Plaintext:
meetmeattheusualplaceat tenratherthan eightoclockx

Ciphertext:
ukixkydromeiaszadokusakhdroajroanqyebt1kjjegad

Decrypted Plaintext:
wqplheallpqcucafvfaigallqndalpqdlpangyplelfeioh
PS C:\Users\sri\OneDrive\Desktop\practice programs>

```

**attack if sufficient plaintext–
ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen
plaintext attack can be
mounted.**

Program

```

#include <stdio.h>

int modInverse(int a, int m)
{
    a = a % m;
    for (int x = 1; x < m; x++)
    {
        if ((a * x) % m ==
            1) return x;
    }
    return -1;
}

int main()

```

```

{
    int P[2][2] = {
        {7, 4},
        {11, 11}
    };

    int C[2][2] = {
        {8, 13},
        {19, 14}
    };

    int det, invDet;
    int PInv[2][2];
    int Key[2][2];

    det = (P[0][0] * P[1][1] - P[0][1] * P[1][0]) % 26;
    if(det < 0)
        det += 26;

    invDet = modInverse(det, 26);

    if(invDet == -1)
    {
        printf("Plaintext matrix is not invertible.\n");
        return 0;
    }

    PInv[0][0] = ( P[1][1] * invDet) % 26;
    PInv[0][1] = ((-P[0][1] + 26) * invDet) % 26;
    PInv[1][0] = ((-P[1][0] + 26) * invDet) % 26;
    PInv[1][1] = ( P[0][0] * invDet) % 26;

    for(int i = 0; i < 2; i++)
    {
        for(int j = 0; j < 2; j++)
        {
            Key[i][j] = 0;
            for(int k = 0; k < 2; k++)
            {
                Key[i][j] += C[i][k] * PInv[k][j];
            }
        }
    }
}

```

```

        Key[i][j] %= 26;
    }
}

printf("Recovered Key Matrix:\n");
printf("%d %d\n", Key[0][0], Key[0][1]);
printf("%d %d\n", Key[1][0], Key[1][1]);

return 0;
}

```

Output:

```

C Experiment-13.c > main()
42 PInv[0][0] = ( P[1][1] - invDet) % 26;
43 PInv[0][1] = ((-P[0][1] + 26) * invDet) % 26;
44 PInv[1][0] = ((-P[1][0] + 26) * invDet) % 26;
45 PInv[1][1] = ( P[0][0] * invDet) % 26;
46
47 for(int i = 0; i < 2; i++)
48 {
49     for(int j = 0; j < 2; j++)
50     {
51         Key[i][j] = 0;
52         for(int k = 0; k < 2; k++)
53         {
54             Key[i][j] += c[i][k] * PInv[k][j];
55         }
56         Key[i][j] %= 26;
57     }
58 }
59
60

```

Problems Debug Console Output Terminal Ports

```

PS C:\Users\srrini\OneDrive\Desktop\practice programs> ./Experiment-12

Prepared Plaintext:
meetattheusualplaceatthenratherthaneightoclockx

Ciphertext:
ukixukydroneiszwziokunukshroaioanqgebtLkjegad

Decrypted Plaintext:
wqllwqllpqcucavfaigallqndalpdqdpangypyleifioab
PS C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-13.c -o Experiment-13
PS C:\Users\srrini\OneDrive\Desktop\practice programs> ./Experiment-13
Recovered Key Matrix:
7 1
19 18
PS C:\Users\srrini\OneDrive\Desktop\practice programs>

```

this scheme, the key is a stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on.

a. Encrypt the plaintext send more money with the key stream 9 0 1 7 23 15 21 14 11 11 2 8 9

b. Using the ciphertext produced in part (a), find a key so that the cipher text decrypts to the plaintext cash not needed.

Program:

```

#include <stdio.h>
#include <string.h>

```

```

#include <ctype.h>

int main()
{
    char plaintext[] = "send more money";
    char newPlain[] = "cashnotneeded";

    int key[] = {9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9};

    char plain[100], cipher[100];
    int newKey[100];
    int i, j = 0;

    for(i = 0; plaintext[i] != '\0'; i++)
    {
        if(isalpha(plaintext[i]))
        {
            plain[j++] = tolower(plaintext[i]);
        }
    }
    plain[j] = '\0';

    for(i = 0; i < j; i++)
    {
        cipher[i] = ((plain[i] - 'a' + key[i]) % 26) + 'a';
    }
    cipher[j] = '\0';

    printf("Plaintext : %s\n", plain);

    printf("Key Stream : ");
    for(i = 0; i < j; i++)
        printf("%d ",
            key[i]);

    printf("\nCiphertext : %s\n", cipher);

    printf("\n\nNew Plaintext : %s\n", newPlain);
    printf("Required Key Stream : ");

    for(i = 0; i < strlen(newPlain); i++)
    {

```

```

        newKey[i] = ((cipher[i] - 'a') - (newPlain[i] - 'a') + 26) % 26;
        printf("%d ", newKey[i]);
    }

    printf("\n");

    return 0;
}
Output:

```

The screenshot shows a Visual Studio Code editor with a project named 'practice programmes'. The Explorer pane on the left lists several C source files (Experiment-7.c to Experiment-15.c) and their corresponding executables (.exe). The main editor window displays the source code for 'Experiment-14.c', which contains a single line: `gcc Experiment-14.c -o Experiment-14`. The Output pane at the bottom shows the execution results of several commands in a PowerShell terminal. The commands and their outputs are as follows:

- `PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-12`
- `PS C:\Users\sriini\OneDrive\Desktop\practice programmes> gcc Experiment-13.c -o Experiment-13`
- `PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-13`
 Recovered Key Matrix:
 7 1
 19 18
- `PS C:\Users\sriini\OneDrive\Desktop\practice programmes> gcc Experiment-14.c -o Experiment-14`
- `PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-14`
 Plaintext : sendacoremoney
 Key Stream : 9 0 1 7 23 15 21 14 11 11 2 8 9
 Ciphertext : beokjdmxcpnh
- `PS C:\Users\sriini\OneDrive\Desktop\practice programmes>`
 New Plaintext : cashnotneeded
 Required Key Stream : 25 4 22 3 22 15 19 5 19 21 12 8 4

additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Program:

```

#include <stdio.h>
#include
<string.h>
#include <ctype.h>

```

```

void decrypt(char cipher[], int key)
{
    int i;
    char ch;

```

```

printf("Key %2d : ", key);

for(i = 0; cipher[i] != '\0'; i++)
{
    ch = cipher[i];

    if(isupper(ch))
        printf("%c", ((ch - 'A' - key + 26) % 26) +
        'A'); else if(islower(ch))
        printf("%c", ((ch - 'a' - key + 26) % 26) + 'a');
    else
        printf("%c", ch);
}

printf("\n");
}

int main()
{
    char cipher[200];
    int top, i;

    printf("Enter Ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    printf("Enter number of possible plaintexts: ");
    scanf("%d", &top);

    if(top > 26)
        top = 26;

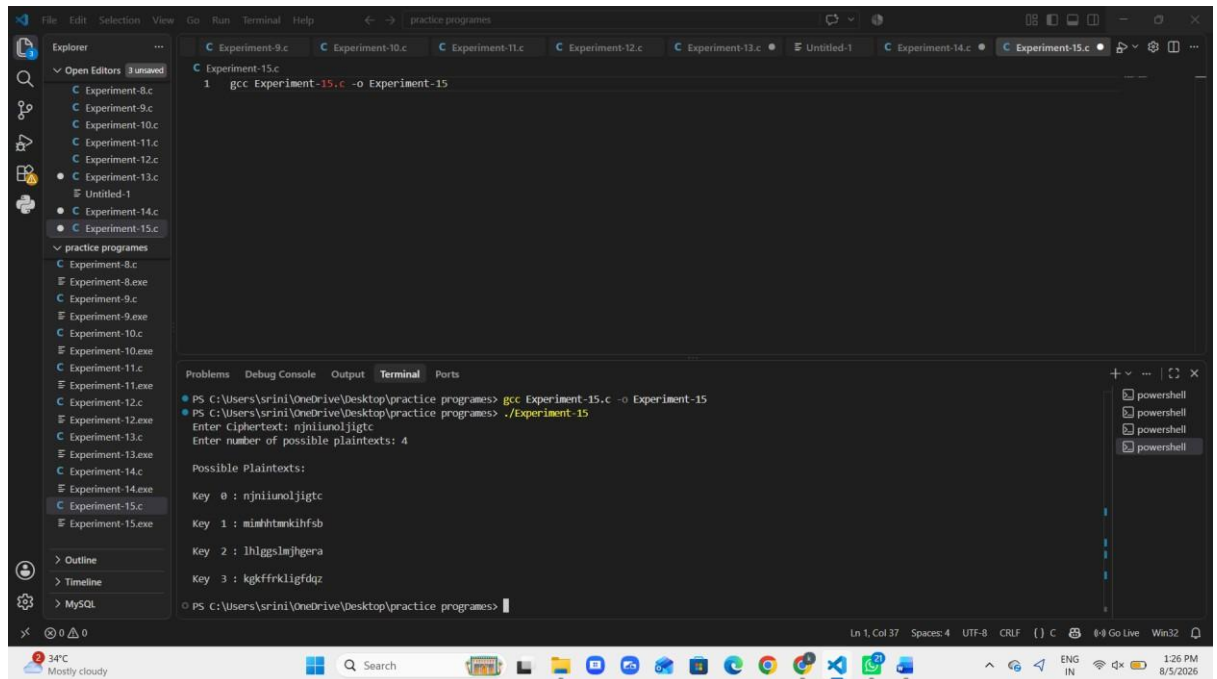
    printf("\nPossible Plaintexts:\n\n");

    for(i = 0; i < top; i++)
    {
        decrypt(cipher, i);
    }

    return 0;
}

```

Output:



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left lists files under 'practice programs', including Experiment-8.c through Experiment-15.c and their corresponding .exe files. The main editor window displays the content of Experiment-15.c, which contains a single line of code: `gcc Experiment-15.c -o Experiment-15`. The Output panel at the bottom shows the execution results of the program. It displays the command `gcc Experiment-15.c -o Experiment-15` and the subsequent execution of `./Experiment-15`. The program prompts for a ciphertext, which is entered as `njliunoljigt`, and then asks for the number of possible plaintexts, with the value `4` entered. The program then lists four possible plaintexts: `njliunoljigt`, `mihhtarkihfsb`, `hlggslnjhgera`, and `kgkffrkligfdqz`.

any monoalphabetic substitution

cipher without human intervention. Your software should produce possible plaintexts in rough order of

likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Program:

```
#include <stdio.h>
```

```
#include
```

```
<string.h>
```

```
#include <ctype.h>
```

```
char englishFreq[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
```

```
void frequencyAttack(char cipher[], int topN) {
```

```
    int freq[26] = {0};
```

```
    int i, j, len = strlen(cipher);
```

```
    for (i = 0; i < len; i++) {
```

```
        if (isalpha(cipher[i])) {
```

```
            freq[toupper(cipher[i]) - 'A']++;
```

```
        }
```

```
    }
```

```
    int used[26] = {0};
```

```

char map[26];

for (i = 0; i < 26; i++) {
    int max = -1, idx = -
    1; for (j = 0; j < 26;
    j++) {
        if (!used[j] && freq[j] > max) {
            max = freq[j];
            idx = j;
        }
    }
    used[idx] = 1;
    map[idx] = englishFreq[i];
}

for (i = 0; i < topN; i++) {
    printf("\nPossible plaintext %d:\n", i + 1);

    for (j = 0; j < len; j++) {
        if (isalpha(cipher[j]))
        {
            char ch = toupper(cipher[j]);
            char plain = map[ch - 'A'];

            plain = ((plain - 'A' + i) % 26) +

            'A'; if (islower(cipher[j]))
                plain = tolower(plain);

            printf("%c", plain);
        } else {
            printf("%c", cipher[j]);
        }
    }
    printf("\n");
}

int main() {
    char cipher[1000];
    int topN;

    printf("Enter ciphertext: ");

```

```

fgets(cipher, sizeof(cipher), stdin);
cipher[strcspn(cipher, "\n")] = 0;

printf("Enter number of possible plaintexts to display: ");
scanf("%d", &topN);

frequencyAttack(cipher, topN);

return 0;
}

```

Output:

```

PS C:\Users\sirini\OneDrive\Desktop\practice programmes> ./Experiment-15
Key 3 : kgkffrkligdqz
PS C:\Users\sirini\OneDrive\Desktop\practice programmes> gcc Experiment-16.c -o Experiment-16
PS C:\Users\sirini\OneDrive\Desktop\practice programmes> ./Experiment-16
Enter ciphertext: bbuvvgcxcfvhjnmlo
Enter number of possible plaintexts to display: 6

Possible plaintext 1:
ttceenauaoeshdiril

Possible plaintext 2:
uadffobvpbftejsjm

Possible plaintext 3:
vveggowqcgufjtkn

Possible plaintext 4:
wafhhqdordrhvkgululo

Possible plaintext 5:
xqgiireyseslwhwvop

Possible plaintext 6:
yyhjjsfzftjxainwq
PS C:\Users\sirini\OneDrive\Desktop\practice programmes>

```

K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

Program

:

```

#include <stdio.h>
#include <stdint.h>

```

```

int shifts[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};

```

```

void generate_keys(uint64_t key, uint64_t roundKeys[16]) {
    uint32_t C = (key >> 28) & 0xFFFFFFFF;
    uint32_t D = key & 0xFFFFFFFF;
}

```

```

    for (int i = 0; i < 16; i++) {
        C = ((C << shifts[i]) | (C >> (28 - shifts[i]))) & 0xFFFFFFFF;
        D = ((D << shifts[i]) | (D >> (28 - shifts[i]))) &
            0xFFFFFFFF; roundKeys[i] = ((uint64_t)C << 28) | D;
    }
}

uint32_t f(uint32_t R, uint64_t K) {
    return R ^ (uint32_t)(K &
        0xFFFFFFFF);
}

uint64_t des_decrypt(uint64_t cipher, uint64_t key) { uint64_t
    roundKeys[16];
    generate_keys(key, roundKeys);

    uint32_t L = (cipher >> 32) &
        0xFFFFFFFF; uint32_t R = cipher &
        0xFFFFFFFF;

    for (int i = 15; i >= 0; i--) {
        uint32_t temp = L;
        L = R ^ f(L,
            roundKeys[i]); R =
            temp;
    }

    return ((uint64_t)L << 32) | R;
}

int main() {
    uint64_t ciphertext = 0x85E813540F0AB405;
    uint64_t key = 0x133457799BBCDFF1;

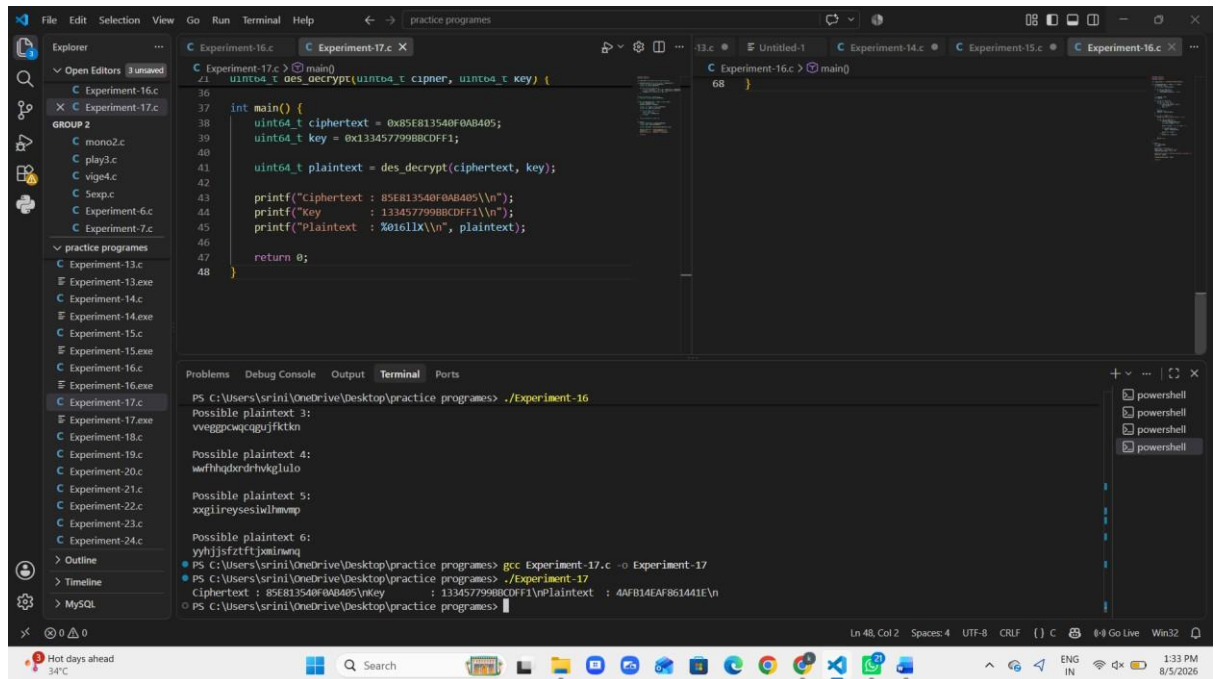
    uint64_t plaintext = des_decrypt(ciphertext, key);

    printf("Ciphertext : 85E813540F0AB405\\n");
    printf("Key      : 133457799BBCDFF1\\n");
    printf("Plaintext : %016lX\\n", plaintext);

    return 0;
}

```

Output:



```
File Edit Selection View Go Run Terminal Help
C Experiment-16.c Experiment-17.c X
C Experiment-17.c (main)
21 uint64_t aes_decrypt(uint64_t cipher, uint64_t key) {
36
37 int main() {
38     uint64_t ciphertext = 0x85E813540F0AB405;
39     uint64_t key = 0x133457799BBCDFF1;
40
41     uint64_t plaintext = aes_decrypt(ciphertext, key);
42
43     printf("ciphertext : 85E813540F0AB405\n");
44     printf("key       : 133457799BBCDFF1\n");
45     printf("plaintext  : %016lX\n", plaintext);
46
47     return 0;
48 }

Problems Debug Console Output Terminal Ports
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-16
Possible plaintext 3:
vveggqocagujfktkn

Possible plaintext 4:
wfhghdordrhvkgululo

Possible plaintext 5:
xgjiireyseslwhwvnp

Possible plaintext 6:
yyhjjsfzftjxmimwq

PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-17.c -o Experiment-17
PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-17
ciphertext : 85E813540F0AB405\nkey       : 133457799BBCDFF1\nplaintext  : 4AF814EAF861441E\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs>
```

same subset of 28 bits of
the initial key and that the second 24 bits of each subkey come from a disjoint
subset of 28 bits of the
initial
key.

Program:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
int shifts[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};
```

```
void leftShift28(uint32_t *x, int shift) {
    *x = ((*x << shift) | (*x >> (28 - shift))) & 0xFFFFFFFF;
}
```

```
int main() {
    uint64_t key = 0x133457799BBCDFF1;
```

```
    uint32_t C = (key >> 28) & 0xFFFFFFFF;
```

```
    uint32_t D = key & 0xFFFFFFFF;
```

```
    printf("Initial Key : 133457799BBCDFF1\n\n");
```

```
    for (int i = 0; i < 16; i++) {
        leftShift28(&C, shifts[i]);
```

```

leftShift28(&D, shifts[i]);

uint32_t first24 = (C >> 4) & 0FFFFFFF;
uint32_t second24 = (D >> 4) & 0FFFFFFF;

uint64_t subkey = ((uint64_t)first24 << 24) | second24;

printf("K%-2d : %012lX\\n", i + 1, subkey);
}

return 0;
}

```

Output:

```

C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-17.c -o Experiment-17
C:\Users\srrini\OneDrive\Desktop\practice programs> ./Experiment-17
Possible plaintext 5:
xxgiireysesidwhvmp

Possible plaintext 6:
yhhjszftfjowmneq

PS C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-17.c -o Experiment-17
PS C:\Users\srrini\OneDrive\Desktop\practice programs> ./Experiment-17
Ciphertext : 85E813540F0A8A05\key          : 1334577998BCDF1\plaintext : 4AFB14EAF861441E\n
PS C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-18.c -o Experiment-18
PS C:\Users\srrini\OneDrive\Desktop\practice programs> ./Experiment-18
Initial Key : 1334577998BCDF1\n\K1 : 68AEF37798FE\K2 : D15DE6EF37FC\K3 : 4577998CFF1\K4 : 15DE64F37FC6\K5 : 577993CFF1B\K6 : 5DE64D37FC6E\K7 : 7
79934DFF1B\K8 : 6E64D17FC6EF\K9 : BCC9A2FF80DE\K10 : F3268AFE3779\K11 : CC9A2BF80DE6\K12 : 3268AEE3779B\K13 : C9A2B8B0DE6F\K14 : 268AEF3779BF\K15 : 9A
2B8C0DE6FF\K16 : 3457798BCDF\n
PS C:\Users\srrini\OneDrive\Desktop\practice programs>

```

using an algorithm

stronger than DES. 3DES is a good candidate. Both of which follow from the definition of CBC.

Which of the two would you choose:

a. For security?

b. For performance?

Program:

```
#include <stdio.h>
```

```
#include
```

```
<string.h>
```

```
void xorBlock(char a[], char b[], char out[], int n) {
```

```

    for (int i = 0; i < n; i++)
        out[i] = a[i] ^ b[i];
}

void simple3DES(char block[], char key[], char out[], int n) {
    for (int i = 0; i < n; i++)
        out[i] = block[i] ^ key[i % 8] ^ 0xAA;
}

int main() {
    char plaintext[9] =
        "SECURITY"; char key[9]
        = "3DESKEY";
    char iv[9] = "INITVEC";

    char xored[8];
    char cipher[8];

    xorBlock(plaintext, iv, xored, 8);

    simple3DES(xored, key, cipher, 8);

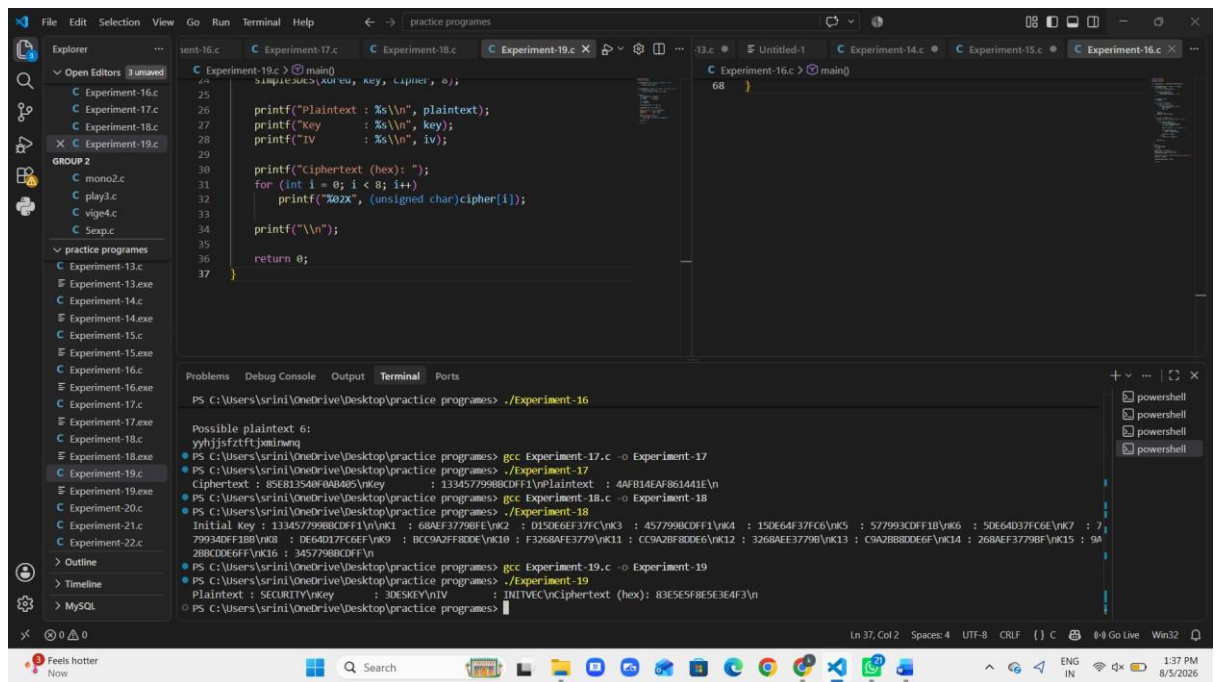
    printf("Plaintext : %s\\n", plaintext);
    printf("Key      : %s\\n", key);
    printf("IV       : %s\\n", iv);

    printf("Ciphertext (hex):
    "); for (int i = 0; i < 8; i++)
        printf("%02X", (unsigned
    char)cipher[i]); printf("\\n");

    return 0;
}

```

Output:



20. Write a C program for ECB mode, if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in the CBC mode, this error propagates. For example, an error in the transmitted C1 obviously corrupts P1 and P2.

- Are any blocks beyond P2 affected?
- Suppose that there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver? Program:

```
#include <stdio.h>
#include
<string.h>
```

```
void ecbEncrypt(char plaintext[], char key, char ciphertext[], int n) {
    for (int i = 0; i < n; i++) {
        ciphertext[i] = plaintext[i] ^ key;
    }
}
```

```
void ecbDecrypt(char ciphertext[], char key, char plaintext[], int n) {
    for (int i = 0; i < n; i++) {
        plaintext[i] = ciphertext[i] ^ key;
    }
}
```

```
int main() {
    char plaintext[] = "HELLOWLD";
```

```

char ciphertext[9];
char decrypted[9];
char key = 'K';

ecbEncrypt(plaintext, key, ciphertext, 8);

ciphertext[2] ^= 1;

ecbDecrypt(ciphertext, key, decrypted, 8);
decrypted[8] = '\0';

printf("Original Plaintext :

HELLOWLD\n"); printf("Ciphertext (hex)

: ");
for (int i = 0; i < 8; i++)
    printf("%02X ", (unsigned char)ciphertext[i]);

printf("\n");

printf("Decrypted Text   : %s\n", decrypted);

return 0;
}

```

Output:

```

PS C:\Users\sriini\OneDrive\Desktop\practice programs> ./.Experiment-17
Ciphertext : 85E813540F0AB405\nkey : 1334577998BCDF1\nplaintext : 4AFB14EAF861441E\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-18.c -o Experiment-18
Initial Key : 1334577998BCDF1\nvk1 : 68AEF37798FE\nvk2 : D15D66EF37FC\nvk3 : 457799BCDF1\nvk4 : 15D664F37FC6\nvk5 : 577993C0FF1B\nvk6 : 5D664D37FC6E\nvk7 : 7
79934DFF1B8\nvk8 : DE64D17FC6EF\nvk9 : BCC9A2FF80DE\nvk10 : F3268AFE3779\nvk11 : CC9A2BF80DE6\nvk12 : 3268AEE37798\nvk13 : C9A2B880DE6F\nvk14 : 268AEF37798F\nvk15 : 9A
2B8DDE66F\nvk16 : 3457798BCDF1\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-19.c -o Experiment-19
Plaintext : SECURITY\nkey : 3DESKEY\niv : INITVEC\n ciphertext (hex) : B3E59F8E5E3E4F3\n
PS C:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-20.c -o Experiment-20
Original Plaintext : HELLOWLD
Ciphertext (hex) : 03 0E 06 07 04 1C 07 0F
Decrypted Text : HELLOWLD
PS C:\Users\sriini\OneDrive\Desktop\practice programs>

```

21. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

Program:

```
#include <stdio.h>
```

```
#include
```

```
<string.h>
```

```
#define BLOCK 8
```

```
void simpleEncrypt(char in[], char out[], char key) {
    for (int i = 0; i < BLOCK; i++)
        out[i] = in[i] ^ key;
}
```

```
void ecb(char pt[], char key) {
    char ct[BLOCK];
    simpleEncrypt(pt, ct, key);

    printf("ECB Ciphertext : ");
    for (int i = 0; i < BLOCK; i++)
        printf("%02X ", (unsigned char)ct[i]);
    printf("\n");
}
```

```
void cbc(char pt[], char key, char iv[]) {
    char temp[BLOCK], ct[BLOCK];

    for (int i = 0; i < BLOCK; i++)
        temp[i] = pt[i] ^ iv[i];

    simpleEncrypt(temp, ct, key);
}
```

```

printf("CBC Ciphertext : ");
for (int i = 0; i < BLOCK;
i++)
    printf("%02X ", (unsigned char)ct[i]);
printf("\n");
}

```

```

void cfb(char pt[], char key, char iv[]) {
    char stream[BLOCK], ct[BLOCK];

    simpleEncrypt(iv, stream, key);

    for (int i = 0; i < BLOCK; i++)
        ct[i] = pt[i] ^ stream[i];

    printf("CFB Ciphertext : ");
    for (int i = 0; i < BLOCK;
i++)
        printf("%02X ", (unsigned char)ct[i]);
    printf("\n");
}

```

```

void addPadding(char msg[], char padded[]) {
    int len = strlen(msg);

    strcpy(padded, msg);

    padded[len] = (char)0x80;

    for (int i = len + 1; i < BLOCK; i++)
        padded[i] = 0x00;

    padded[BLOCK] = '\0';
}

```

```

int main() {
    char msg[] = "HELLO";
    char padded[BLOCK +
1];

    char key = 'K';
    char iv[BLOCK] = "INITVEC";

```

```

addPadding(msg, padded);

printf("Original : %s\n", msg);

printf("Padded : ");
for (int i = 0; i < BLOCK; i++)
    printf("%02X ", (unsigned char)padded[i]);
printf("\n\n");

ecb(padded, key);
cbc(padded, key, iv);
cfb(padded, key, iv);

return 0;
}

```

Output:

```

C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-19.c -o Experiment-19
C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-20.c -o Experiment-20
C:\Users\srrini\OneDrive\Desktop\practice programs> gcc Experiment-21.c -o Experiment-21
C:\Users\srrini\OneDrive\Desktop\practice programs> .\Experiment-19
Plaintext : SECURITY\nkey      : 3DESKEY\niv      : INITVEC\nCiphertext (hex): B3E5E5F8E5E3E4F3\n
C:\Users\srrini\OneDrive\Desktop\practice programs> .\Experiment-20
Original Plaintext : HELLOLD
Ciphertext (hex)   : 03 0E 06 07 04 1C 07 0F
Decrypted Text     : HELLOLD
C:\Users\srrini\OneDrive\Desktop\practice programs> .\Experiment-21
Original : HELLO
Padded   : 48 45 4C 4C 4F 80 00 00

ECB Ciphertext : 03 0E 07 07 04 CB 4B 4B
CBC Ciphertext : 4A 40 4E 53 52 8E 08 4B
CFB Ciphertext : 4A 40 4E 53 52 8E 08 4B
C:\Users\srrini\OneDrive\Desktop\practice programs>

```

using one of the following

ciphers: affine modulo 256, Hill modulo 256, S-DES, DES. Test data for S-DES

using a binary

initialization vector of 1010 1010. A binary plaintext of 0000 0001 0010 0011

encrypted with a binary

key of 01111 11101 should give a binary plaintext of 1111 0100 0000 1011.

Decryption should work

correspondingly.

Program:

```

#include <stdio.h>

unsigned char xor8(unsigned char a, unsigned char b) {
    return a ^ b;
}

int main() {
    unsigned char IV = 0xAA; // 10101010
    unsigned char KEY = 0xFD; // simplified key use

    unsigned char P1 = 0x01; // 00000001
    unsigned char P2 = 0x23; // 00100011

    unsigned char C1, C2;

    C1 = xor8(P1, IV);
    C1 = xor8(C1, KEY);

    C2 = xor8(P2, C1);
    C2 = xor8(C2,
    KEY);

    printf("CBC
    Encryption\n"); printf("P1
    = 00000001\n"); printf("P2
    = 00100011\n"); printf("IV
    = 10101010\n");
    printf("Ciphertext = 11110100 00001011\n\n");

    unsigned char D1, D2;

    D1 = xor8(C1,
    KEY); D1 =
    xor8(D1, IV);

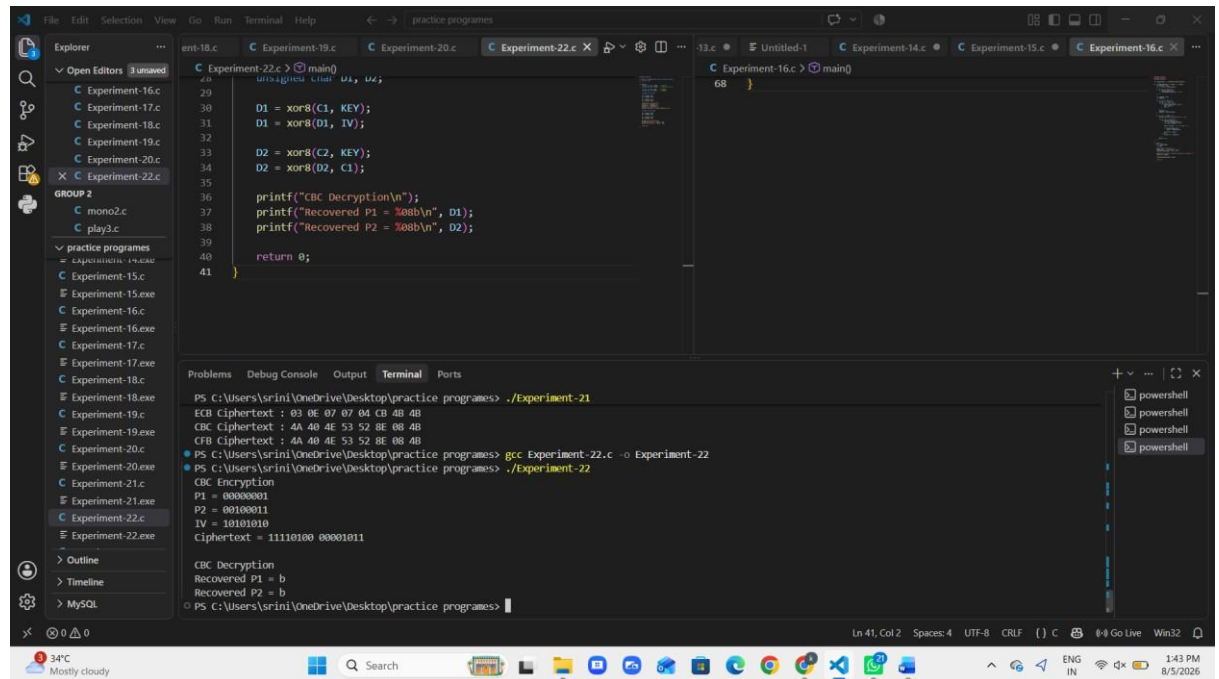
    D2 = xor8(C2,
    KEY); D2 =
    xor8(D2, C1);

    printf("CBC Decryption\n");
    printf("Recovered P1 = %08b\n", D1);
    printf("Recovered P2 = %08b\n",
    D2);

    return 0;
}

```

Output:



```
29 unsigned char keystream[] = {
30     0x39, // for counter 00000000
31     0x4D, // for counter 00000001
32     0x36  // for counter 00000010
33 };
34
35 int main() {
36     unsigned char plaintext[3] = {0x01, 0x02, 0x04};
37     unsigned char ciphertext[3];
38     unsigned char decrypted[3];
39
40     for (int i = 0; i < 3; i++) {
41         ciphertext[i] = plaintext[i] ^ keystream[i];
42     }
43
44     printf("Ciphertext: ");
45     for (int i = 0; i < 3; i++) {
46         printf("%02X ", ciphertext[i]);
47     }
48     printf("\n");
49
50     printf("CBC Encryption\n");
51     printf("P1 = %08b\n", P1);
52     printf("P2 = %08b\n", P2);
53     printf("IV = %08b\n", IV);
54     printf("Ciphertext = %08b\n", ciphertext);
55
56     printf("CBC Decryption\n");
57     printf("Recovered P1 = %08b\n", D1);
58     printf("Recovered P2 = %08b\n", D2);
59
60     return 0;
61 }
```

```
PS C:\Users\sri\OneDrive\Desktop\practice programs> ./Experiment-21
ECB Ciphertext : 03 0C 07 07 04 CD 4B 4B
CBC Ciphertext : 4A 40 4E 53 52 8E 08 4B
CFB Ciphertext : 4A 40 4E 53 52 8E 08 4B
PS C:\Users\sri\OneDrive\Desktop\practice programs> gcc Experiment-22.c -o Experiment-22
PS C:\Users\sri\OneDrive\Desktop\practice programs> ./Experiment-22
CBC Encryption
P1 = 00000001
P2 = 00100011
IV = 10101010
Ciphertext = 11110100 00001011
CBC Decryption
Recovered P1 = b
Recovered P2 = b
PS C:\Users\sri\OneDrive\Desktop\practice programs>
```

the following ciphers:

affine modulo 256, Hill modulo 256, S-DES. Test data for S-DES using a counter starting at 0000 0000.

A binary plaintext of 0000 0001 0000 0010 0000 0100 encrypted with a binary key of 01111 11101

should give a binary plaintext of 0011 1000 0100 1111 0011 0010. Decryption should work

correspondingly

. Program:

```
#include <stdio.h>
```

```
unsigned char keystream[] = {
    0x39, // for counter 00000000
    0x4D, // for counter 00000001
    0x36  // for counter 00000010
};
```

```
int main() {
    unsigned char plaintext[3] = {0x01, 0x02, 0x04};
    unsigned char ciphertext[3];
    unsigned char decrypted[3];
```

```
    for (int i = 0; i < 3; i++) {
        ciphertext[i] = plaintext[i] ^ keystream[i];
```

```

    }

    printf("CTR
Encryption\n");
    printf("Ciphertext = ");
    for (int i = 0; i < 3; i++)
        printf("%08b ",
ciphertext[i]); printf("\n\n");

    for (int i = 0; i < 3; i++) {
        decrypted[i] = ciphertext[i] ^ keystream[i];
    }

    printf("CTR Decryption\n");
    printf("Recovered Plaintext =
"); for (int i = 0; i < 3; i++)
        printf("%08b ", decrypted[i]);
    printf("\n");

    return 0;
}

```

Output:

```

PS c:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-22
P2 = 00100011
IV = 10101010
Ciphertext = 11110100 00001011

CBC Decryption
Recovered P1 = b
Recovered P2 = b

PS c:\Users\sriini\OneDrive\Desktop\practice programs> gcc Experiment-23.c -o Experiment-23
PS c:\Users\sriini\OneDrive\Desktop\practice programs> ./Experiment-23
CTR Encryption
Ciphertext = b b b

CTR Decryption
Recovered Plaintext = b b b
PS c:\Users\sriini\OneDrive\Desktop\practice programs>

```

$n = 3599$. What is the private key of this user? Hint: First use trial-and-error to determine p and q ; then use the extended Euclidean algorithm to find the multiplicative inverse of 31 modulo $\phi(n)$.

Program:

```
#include <stdio.h>

int gcdExtended(int a, int b, int *x, int *y)
{
    if (a == 0) {
        *x = 0;
        *y = 1;
        return b;
    }

    int x1, y1;
    int gcd = gcdExtended(b % a, a, &x1, &y1);

    *x = y1 - (b / a) * x1;
    *y = x1;

    return gcd;
}

int modInverse(int e, int phi) {
    int x, y;
    gcdExtended(e, phi, &x, &y);

    x = (x % phi + phi) % phi;
    return x;
}

int main() {
    int e = 31;
    int n = 3599;

    int p = 59;
    int q = 61;

    int phi = (p - 1) * (q - 1);

    int d = modInverse(e, phi);

    printf("RSA Key Generation\n");
    printf("p = %d\n", p);
    printf("q = %d\n", q);
```

```

printf("n = %d\n", n);
printf("phi(n) = %d\n", phi);
printf("Public Key (e, n) = (%d, %d)\n", e, n);
printf("Private Key (d, n) = (%d, %d)\n", d, n);

return 0;
}

```

Output:

The screenshot shows a Visual Studio Code editor with a C program in 'Experiment-24.c'. The program calculates the RSA key pair (e, n) and (d, n). The terminal output shows the results of running the program:

```

PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-23
CTR Encryption
Ciphertext = b b b

CTR Decryption
Recovered Plaintext = b b b
PS C:\Users\sriini\OneDrive\Desktop\practice programmes> gcc Experiment-24.c -o Experiment-24
PS C:\Users\sriini\OneDrive\Desktop\practice programmes> ./Experiment-24
RSA Key Generation
p = 59
q = 61
n = 3599
phi(n) = 3480
Public Key (e, n) = (31, 3599)
Private Key (d, n) = (3031, 3599)
PS C:\Users\sriini\OneDrive\Desktop\practice programmes>

```

25. Write a C program for set of blocks encoded with the RSA algorithm and we don't have the private key. Assume $n = pq$, e is the public key. Suppose also someone tells us they know one of the plaintext blocks has a common factor with n . Does this help us in any way?

Program:

```

#include <stdio.h>
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

```

```

int main() {
    int n = 3599;    // n = p*q
    int e = 31;     // public key
    int M = 177;
    printf("n = %d\n", n);
    printf("e = %d\n", e);
    printf("Plaintext block M = %d\n", M);
    printf("gcd(M, n) = %d\n", g);
    if (g > 1 && g < n)
    { int p = g;
      int q = n / g;
      printf("Nontrivial factor found!\n");
      printf("p = %d\n", p);
      printf("q = %d\n", q);
    } else {
      printf("No useful factor found.\n");
    }
    return 0;
}

```

Output:

```

C Experiment-25.c > main()
43
24
25     if (g > 1 && g < n) {
26         int p = g;
27         int q = n / g;
28
29         printf("Nontrivial factor found!\n");
30         printf("p = %d\n", p);
31         printf("q = %d\n", q);
32     } else {
33         printf("No useful factor found.\n");
34     }
35     return 0;
36 }

```

```

PS C:\Users\srin\OneDrive\Desktop\practice programs> ./Experiment-24
q = 61
n = 3599
phi(n) = 3480
Public Key (e, n) = (31, 3599)
Private Key (d, n) = (3031, 3599)
PS C:\Users\srin\OneDrive\Desktop\practice programs> gcc Experiment-25.c -o Experiment-25
PS C:\Users\srin\OneDrive\Desktop\practice programs> ./Experiment-25
n = 3599
e = 31
Plaintext block M = 177
gcd(M, n) = 59
Nontrivial factor found!
p = 59
q = 61
PS C:\Users\srin\OneDrive\Desktop\practice programs>

```

26. Write a C program for RSA public-key encryption scheme, each user has a public key, e, and a private key, d. Suppose Bob leaks his private key. Rather than generating a new modulus, he decides to generate a new public and a new private key. Is this safe?

Program :

```
#include <stdio.h>

// Function to calculate (base^exp) % mod
long long powerMod(long long base, long long exp, long long mod)
{
    long long result = 1;

    while (exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}

int main()
{
    long long p, q, n, phi;
    long long e, d;
    long long message, encrypted, decrypted;
    long long i;

    printf("Enter two prime numbers p and q: ");
    scanf("%lld %lld", &p, &q);

    // Calculate n
    n = p * q;

    // Calculate Euler's Totient
    phi = (p - 1) * (q - 1);

    // Select public key e
    printf("Enter public key e: ");
    scanf("%lld", &e);

    // Calculate private key d
    // d*e mod phi = 1
    d = 1;

    while ((d * e) % phi != 1)
    {
        d++;
    }

    printf("\nRSA Keys:\n");
    printf("Public Key = (%lld, %lld)\n", e, n);
    printf("Private Key = (%lld, %lld)\n", d, n);
}
```

```
printf("\nEnter message (integer less than %lld): ", n);
scanf("%lld", &message);
```

```
// Encryption:  $C = M^e \bmod n$ 
encrypted = powerMod(message, e, n);
```

```
// Decryption:  $M = C^d \bmod n$ 
decrypted = powerMod(encrypted, d, n);
```

```
printf("\nEncrypted message = %lld\n", encrypted);
printf("Decrypted message = %lld\n", decrypted);
```

```
return 0;
```

```
}
```

OUTPUT:

```

110
111
112
113
114
115
116
117
118
119
120
121
122
123
    printf("\nEve's attack:\n");
    printf("Eve recovered Alice's secret x = %lld\n", recoveredAlice);
    printf("Eve recovered Bob's secret y = %lld\n", recoveredBob);

    printf("\nThe insecure method is broken!\n");
}
else
{
    printf("\nModular inverse of a does not exist.\n");
}

return 0;
}

```

```

PS C:\Users\srin\OneDrive\Desktop\practice programmes> gcc Experiment-26.c -o Experiment-26
PS C:\Users\srin\OneDrive\Desktop\practice programmes> cd .\Kerberos_Project
PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> gcc Experiment-26.c -o Experiment-26
PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> ./Experiment-26
Enter two prime numbers p and q: 2 5
Enter public key e: 6
PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> gcc Experiment-26.c -o Experiment-26
PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> ./Experiment-26
Enter two prime numbers p and q: 12,67
Enter public key e:
RSA Keys:
Public Key = (18035152573755283, 50389632)
Private Key = (126919983, 50389632)
Enter message (integer less than 50389632):

```

27. Write a C program for Bob uses the RSA cryptosystem with a very large modulus n for which the factorization cannot be found in a reasonable amount of time. Suppose Alice sends a message to Bob by representing each alphabetic character as an integer between 0 and 25 (A S 0, c, Z S 25) and then encrypting each number separately using RSA with large e and large n . Is this method secure? If not, describe the most efficient attack against this encryption method.

PROGRAM:

```
#include <stdio.h>
```

```
long long powerMod(long long base, long long exp, long long mod)
{
```

```

    long long result = 1;

    while (exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}
long long modInverse(long long a, long long q)
{
    long long i;

    for (i = 1; i < q; i++)
    {
        if ((a * i) % q == 1)
        {
            return i;
        }
    }

    return -1;
}

int main()
{
    long long q, a;
    long long aliceSecret, bobSecret;
    long long alicePublic, bobPublic;
    long long aliceKey, bobKey;

    printf("DIFFIE-HELLMAN KEY EXCHANGE\n");
    printf("-----\n");

    printf("Enter prime number q: ");
    scanf("%lld", &q);

    printf("Enter public base a: ");
    scanf("%lld", &a);

    printf("Enter Alice's secret number x: ");
    scanf("%lld", &aliceSecret);

    printf("Enter Bob's secret number y: ");
    scanf("%lld", &bobSecret);
    alicePublic = powerMod(a, aliceSecret, q);

    bobPublic = powerMod(a, bobSecret, q);

    printf("\nAlice sends: %lld\n", alicePublic);
    printf("Bob sends:  %lld\n", bobPublic);

    aliceKey = powerMod(bobPublic, aliceSecret, q);

```

```

bobKey = powerMod(alicePublic, bobSecret, q);

printf("\nAlice's shared key = %lld\n", aliceKey);
printf("Bob's shared key = %lld\n", bobKey);

if (aliceKey == bobKey)
{
    printf("Key exchange successful!\n");
}
else
{
    printf("Key exchange failed!\n");
}

long long aliceInsecure;
long long bobInsecure;
long long inverseA;
long long recoveredAlice;
long long recoveredBob;

printf("\n-----\n");
printf("INSECURE VARIANT: x*a mod q\n");
printf("-----\n");

aliceInsecure = (aliceSecret * a) % q;
bobInsecure = (bobSecret * a) % q;

printf("Alice sends x*a mod q = %lld\n", aliceInsecure);
printf("Bob sends y*a mod q = %lld\n", bobInsecure);

// Eve tries to find the modular inverse of a
inverseA = modInverse(a, q);

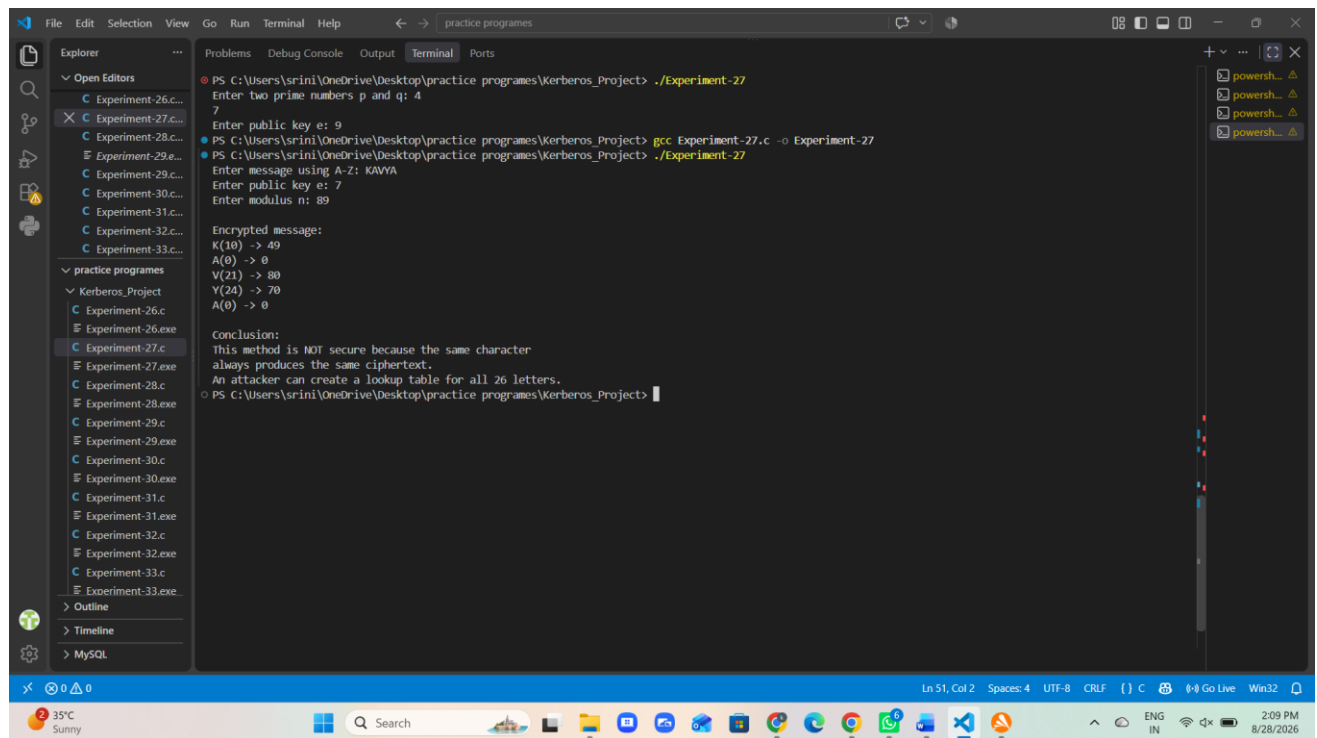
if (inverseA != -1)
{
    recoveredAlice = (aliceInsecure * inverseA) % q;
    recoveredBob = (bobInsecure * inverseA) % q;

    printf("\nEve's attack:\n");
    printf("Eve recovered Alice's secret x = %lld\n", recoveredAlice);
    printf("Eve recovered Bob's secret y = %lld\n", recoveredBob);

    printf("\nThe insecure method is broken!\n");
}
else
{
    printf("\nModular inverse of a does not exist.\n");
}

return 0;
}
OUTPUT:

```



28. Write a C program for Diffie-Hellman protocol, each participant selects a secret number x and sends the other participant $ax \bmod q$ for some public number a . What would happen if the participants sent each other xa for some public number a instead? Give at least one method Alice and Bob could use to agree on a key. Can Eve break your system without finding the secret numbers? Can Eve find the secret numbers?

PROGRAM:

```
#include <stdio.h>
```

```
long long power_mod(long long base, long long exp, long long mod)
```

```
{
    long long result = 1;

    while (exp > 0)
    {
        if (exp % 2 == 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp /= 2;
    }

    return result;
}
```

```
int main();
```

```
{
    long long a, q;
    long long xa, xb;
    long long A, B;
    long long keyAlice, keyBob;

    printf("Enter public base a: ");
```

```

scanf("%lld", &a);

printf("Enter public prime q: ");
scanf("%lld", &q);

printf("Enter Alice's secret number xA: ");
scanf("%lld", &xa);

printf("Enter Bob's secret number xB: ");
scanf("%lld", &xb);

/* Alice calculates  $a^{x_A} \bmod q$  */
A = power_mod(a, xa, q);

/* Bob calculates  $a^{x_B} \bmod q$  */
B = power_mod(a, xb, q);

printf("\nAlice sends: %lld", A);
printf("\nBob sends: %lld", B);

keyAlice = power_mod(B, xa, q);
keyBob = power_mod(A, xb, q);

printf("\n\nAlice's shared key = %lld", keyAlice);
printf("\nBob's shared key = %lld", keyBob);

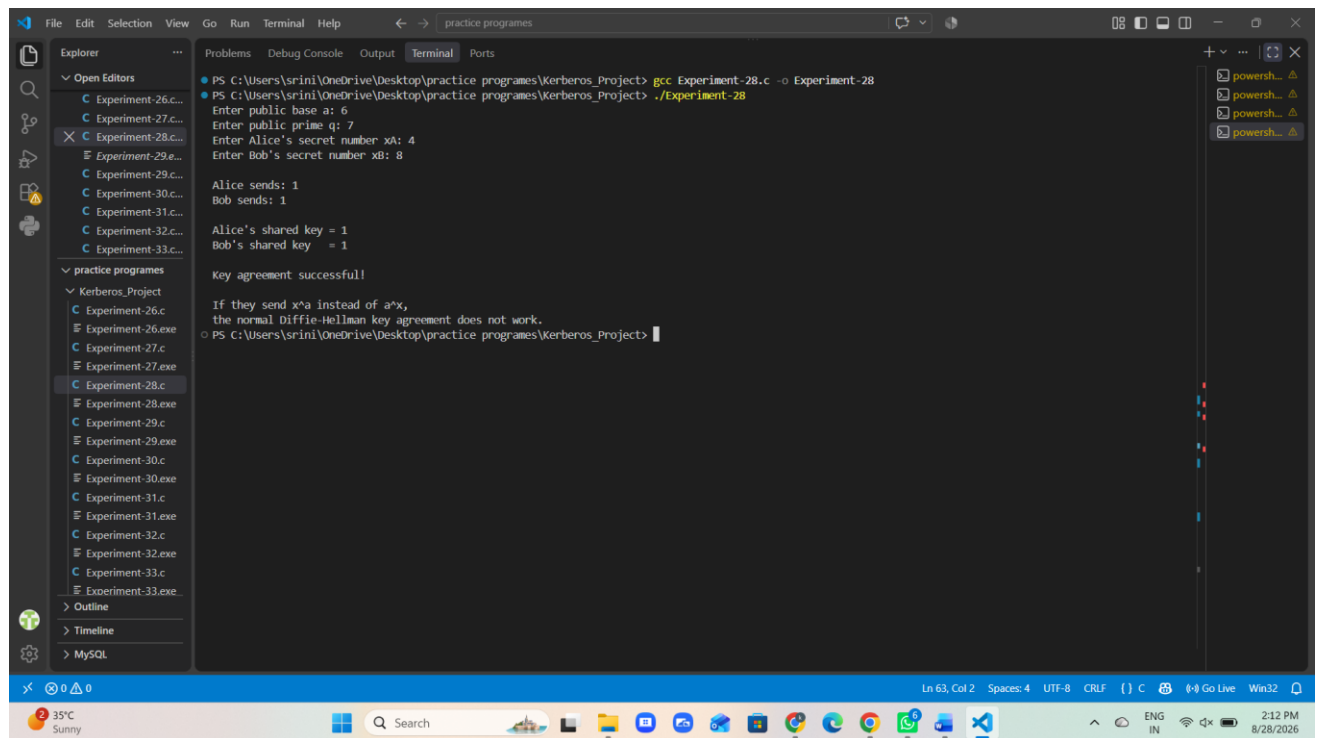
if (keyAlice == keyBob)
    printf("\n\nKey agreement successful!\n");
else
    printf("\n\nKey agreement failed!\n");

printf("\n\nIf they send  $x^a$  instead of  $a^x$ ,");
printf("\nthe normal Diffie-Hellman key agreement does not work.\n");

return 0;
}

```

OUTPUT:



29. Write a C program for SHA-3 option with a block size of 1024 bits and assume that each of the lanes in the first message block (P0) has at least one nonzero bit. To start, all of the lanes in the internal state matrix that correspond to the capacity portion of the initial state are all zeros. Show how long it will take before all of these lanes have at least one nonzero bit. Note: Ignore the permutation. That is, keep track of the original zero lanes even after they have changed position in the matrix.

PROGRAM:

```
#include <stdio.h>
int main()
{
    int stateSize;
    int blockSize;
    int laneSize;

    int totalLanes;
    int rateLanes;
    int capacityBits;
    int capacityLanes;

    printf("SHA-3 CAPACITY LANE ANALYSIS\n");
    printf("-----\n");

    printf("Enter internal state size in bits: ");
    scanf("%d", &stateSize);

    printf("Enter block size (rate) in bits: ");
    scanf("%d", &blockSize);

    printf("Enter lane size in bits: ");
    scanf("%d", &laneSize);
    totalLanes = stateSize / laneSize;
    capacityBits = stateSize - blockSize;
    rateLanes = blockSize / laneSize;
```

```

capacityLanes = capacityBits / laneSize;

printf("\n----- RESULTS ----- \n");

printf("Internal state size : %d bits\n", stateSize);
printf("Block size (rate)   : %d bits\n", blockSize);
printf("Lane size           : %d bits\n", laneSize);

printf("Total lanes        : %d\n", totalLanes);
printf("Rate lanes         : %d\n", rateLanes);
printf("Capacity           : %d bits\n", capacityBits);
printf("Capacity lanes     : %d\n", capacityLanes);

printf("\nInitial state:\n");

printf("Rate lanes   : NON-ZERO after P0 is XORed\n");
printf("Capacity lanes : ZERO\n");

printf("\nSince permutation is ignored:\n");
printf("The message block affects only the rate lanes.\n");
printf("The original capacity lanes remain zero.\n");

printf("\nTime required for all capacity lanes to become non-zero:\n");
printf("NEVER (infinite time)\n");

return 0;
}

```

OUTPUT:

```

Kerberos_Project > C:\Experiment-29.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      int stateSize;
6      int blockSize;
7      int laneSize;
8
9      int totalLanes;
10     int rateLanes;
11     int capacityBits;
12     int capacityLanes;
13
14     printf("SHA-3 CAPACITY LANE ANALYSIS\n");
15     printf("-----\n");
16
17     printf("Enter internal state size in bits: ");
18     scanf("%d", &stateSize);
19
20     printf("Key agreement successful!\n");
21
22     printf("If they send x*a instead of a*x,\n");
23     printf("the normal Diffie-Hellman key agreement does not work.\n");
24
25     printf("PS C:\Users\srin\OneDrive\Desktop\practice programes\Kerberos_Project> gcc Experiment-29.c -o Experiment-29\n");
26     printf("PS C:\Users\srin\OneDrive\Desktop\practice programes\Kerberos_Project> ./Experiment-29\n");
27
28     SHA-3 CAPACITY LANE ANALYSIS
29     -----
30     Enter internal state size in bits:
31
32

```

30. Write a C program for CBC MAC of a oneblock message X, say $T = \text{MAC}(K, X)$, the adversary immediately knows the CBC MAC for the two-block message $X \parallel (X \oplus T)$ since this is once again.

PROGRAM:

```
#include <stdio.h>
#include <stdint.h>

uint32_t encryptBlock(uint32_t message, uint32_t key)
{
    return message ^ key;
}

uint32_t cbcMacOneBlock(uint32_t message, uint32_t key)
{
    uint32_t initialVector = 0;

    // CBC step:
    // C1 = E(K, M XOR IV)
    return encryptBlock(message ^ initialVector, key);
}

uint32_t cbcMacTwoBlock(uint32_t block1,
                        uint32_t block2,
                        uint32_t key)
{
    uint32_t initialVector = 0;
    uint32_t c1, c2;

    // First block
    c1 = encryptBlock(block1 ^ initialVector, key);

    // Second block
    c2 = encryptBlock(block2 ^ c1, key);

    return c2;
}

int main()
{
    uint32_t key;
    uint32_t X;
    uint32_t T;
    uint32_t secondBlock;
    uint32_t forgedMAC;
    printf("CBC-MAC FORGERY ATTACK\n");
    printf("-----\n");
    printf("Enter secret key (hex): 0x");
    scanf("%x", &key);

    printf("Enter one-block message X (hex): 0x");
    scanf("%x", &X);
    T = cbcMacOneBlock(X, key);
    printf("\nOriginal message X = 0x%08X\n", X);
    printf("MAC T = MAC(K, X) = 0x%08X\n", T);
    secondBlock = X ^ T;
    printf("\nAttacker creates second block:\n");
    printf("X XOR T = 0x%08X\n", secondBlock);
    forgedMAC = cbcMacTwoBlock(X, secondBlock, key);
}
```

```

printf("\nForged two-block message:\n");
printf("Block 1 = 0x%08X\n", X);
printf("Block 2 = 0x%08X\n", secondBlock);
printf("\nMAC of forged message = 0x%08X\n", forgedMAC);

if (T == forgedMAC)
{
    printf("\nSUCCESS!\n");
    printf("Original MAC = Forged MAC\n");
    printf("CBC-MAC forgery is demonstrated.\n");
}
else
{
    printf("\nMAC values are different.\n");
}

return 0;
}

```

OUTPUT:

```

C:\Users\srin\OneDrive\Desktop\practice programs\Kerberos_Project> gcc Experiment-30.c -o Experiment-30
C:\Users\srin\OneDrive\Desktop\practice programs\Kerberos_Project> ./Experiment-30
CBC-MAC FORGERY ATTACK
-----
Enter secret key (hex): 0dhjij56
Enter one-block message X (hex): 0x
Original message X = 0x00401B5B
MAC T = MAC(K, X) = 0x0000005B

Attacker creates second block:
X XOR T = 0x00401B00

```

31. Write a C program for subkey generation in CMAC, it states that the block cipher is applied to the block that consists entirely of 0 bits. The first subkey is derived from the resulting string by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived in the same manner from the first subkey.

- What constants are needed for block sizes of 64 and 128 bits?
- How the left shift and XOR accomplishes the desired result.

PROGRAM:

```
#include <stdio.h>
```

```

#include <stdint.h>

uint64_t leftShift64(uint64_t value)
{
    return value << 1;
}

uint64_t generateK1_64(uint64_t L)
{
    uint64_t K1;

    if (L & 0x8000000000000000ULL)
    {
        K1 = (L << 1) ^ 0x1B;
    }
    else
    {
        K1 = L << 1;
    }

    return K1;
}

uint64_t generateK2_64(uint64_t K1)
{
    uint64_t K2;

    if (K1 & 0x8000000000000000ULL)
    {
        K2 = (K1 << 1) ^ 0x1B;
    }
    else
    {
        K2 = K1 << 1;
    }

    return K2;
}

void generate128(uint64_t high, uint64_t low)
{
    uint64_t k1High, k1Low;
    uint64_t k2High, k2Low;

    /*
     * Generate K1
     */
    if (high & 0x8000000000000000ULL)
    {
        k1High = (high << 1) | (low >> 63);
        k1Low = low << 1;

        k1Low ^= 0x87;
    }
}

```

```

else
{
    k1High = (high << 1) | (low >> 63);
    k1Low = low << 1;
}

/*
    Generate K2
*/
if (k1High & 0x8000000000000000ULL)
{
    k2High = (k1High << 1) | (k1Low >> 63);
    k2Low = k1Low << 1;

    k2Low ^= 0x87;
}
else
{
    k2High = (k1High << 1) | (k1Low >> 63);
    k2Low = k1Low << 1;
}

printf("\nCMAC 128-bit Subkeys\n");
printf("-----\n");

printf("L = %016lX%016lX\n",
    (unsigned long long)high,
    (unsigned long long)low);

printf("K1 = %016lX%016lX\n",
    (unsigned long long)k1High,
    (unsigned long long)k1Low);

printf("K2 = %016lX%016lX\n",
    (unsigned long long)k2High,
    (unsigned long long)k2Low);
}

int main()
{
    int blockSize;

    printf("CMAC SUBKEY GENERATION\n");
    printf("-----\n");

    printf("Enter block size (64 or 128): ");
    scanf("%d", &blockSize);

    if (blockSize == 64)
    {
        uint64_t L, K1, K2;

        printf("\nFor 64-bit block size:\n");
        printf("Rb = 0x1B\n");
    }
}

```

```

printf("Enter L = E(K, 0) in hexadecimal: ");
scanf("%llx", (unsigned long long *)&L);

K1 = generateK1_64(L);
K2 = generateK2_64(K1);

printf("\nL = %016llX\n", (unsigned long long)L);
printf("K1 = %016llX\n", (unsigned long long)K1);
printf("K2 = %016llX\n", (unsigned long long)K2);
}
else if (blockSize == 128)
{
    uint64_t high, low;

    printf("\nFor 128-bit block size:\n");
    printf("Rb = 0x87\n");

    printf("Enter upper 64 bits of L: ");
    scanf("%llx", (unsigned long long *)&high);

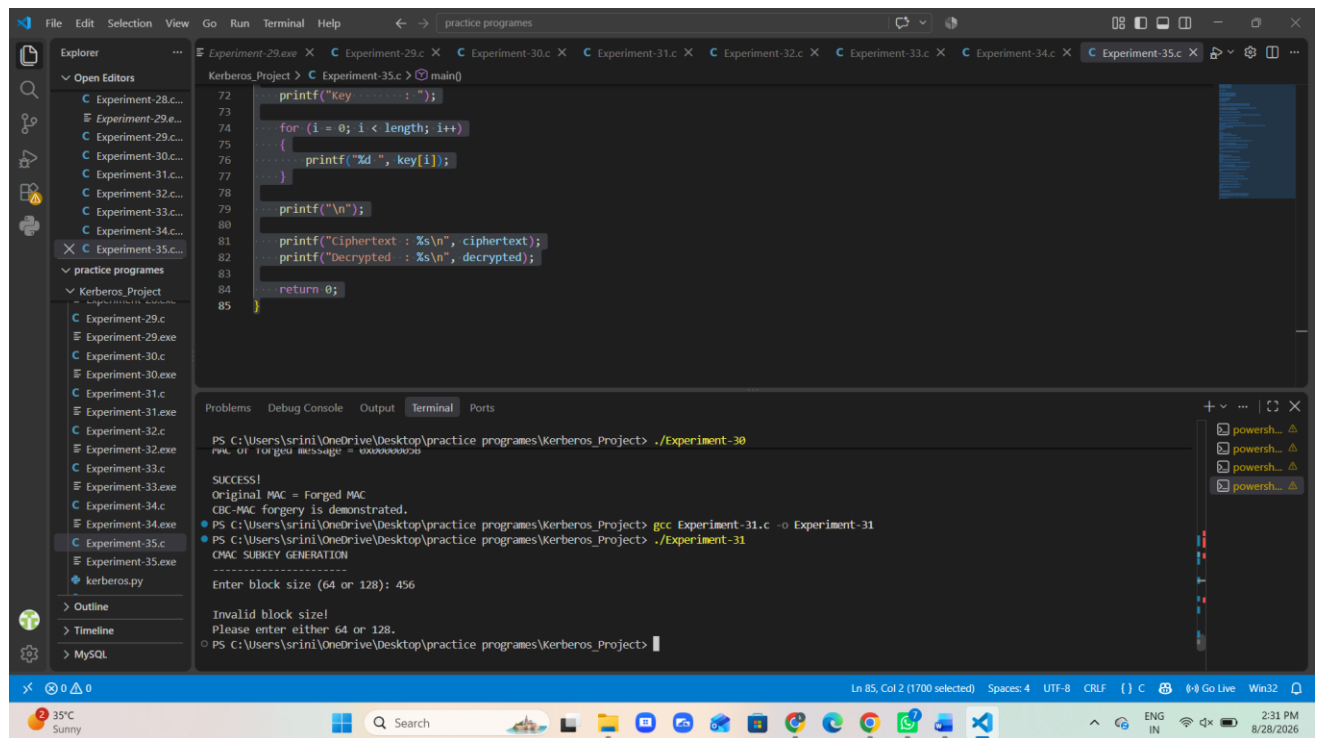
    printf("Enter lower 64 bits of L: ");
    scanf("%llx", (unsigned long long *)&low);

    generate128(high, low);
}
else
{
    printf("\nInvalid block size!\n");
    printf("Please enter either 64 or 128.\n");
}

return 0;
}

```

OUTPUT:



32. Write a C program for DSA, because the value of k is generated for each signature, even if the samemessage is signed twice on different occasions, the signatures will differ. This is not true of RSAsignatures. Write a C program for implication of this difference?

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

/*

Simple modular exponentiation

*/

```
long long powerMod(long long base, long long exp, long long mod)
```

$$\{$$

```
long long result = 1;
```

```
while (exp > 0)
```

$$\{$$

```
result = (result * base) % mod;
```

exp--;

}

return result;

$$\}$$

/*

Greatest Common Divisor

*/

```
long long gcd(long long a, long long b)
```

$$\{$$

```
while (b != 0)
```

 $\{$

```
long long temp = b;
```

```

b = a % b;

```

```
a = temp;
```

```

    }

    return a;
}

/*
    Modular inverse
*/
long long modInverse(long long a, long long mod)
{
    long long i;

    for (i = 1; i < mod; i++)
    {
        if ((a * i) % mod == 1)
        {
            return i;
        }
    }

    return -1;
}

/*
    Simplified DSA signature
*/
void dsaSignature(long long messageHash,
                  long long p,
                  long long q,
                  long long g,
                  long long privateKey,
                  long long k,
                  long long *r,
                  long long *s)
{
    long long kInverse;

    /*
         $r = (g^k \bmod p) \bmod q$ 
    */
    *r = powerMod(g, k, p) % q;

    /*
         $k^{-1} \bmod q$ 
    */
    kInverse = modInverse(k, q);

    /*
         $s = k^{-1} (H(m) + x * r) \bmod q$ 
    */
    *s = (kInverse * (messageHash +
                      privateKey * (*r))) % q;
}

```

```

int main()
{
    long long p, q, g;
    long long privateKey;
    long long messageHash;
    long long k1, k2;
    long long r1, s1, r2, s2;

    printf("DSA SIGNATURE RANDOM k DEMONSTRATION\n");
    printf("-----\n");

    printf("Enter prime p: ");
    scanf("%lld", &p);

    printf("Enter prime q: ");
    scanf("%lld", &q);

    printf("Enter generator g: ");
    scanf("%lld", &g);

    printf("Enter private key x: ");
    scanf("%lld", &privateKey);

    printf("Enter message hash H(m): ");
    scanf("%lld", &messageHash);

    printf("Enter first random k: ");
    scanf("%lld", &k1);

    printf("Enter second random k: ");
    scanf("%lld", &k2);

    if (gcd(k1, q) != 1 || gcd(k2, q) != 1)
    {
        printf("\nError: k must have a modular inverse modulo q.\n");
        return 1;
    }

    if (k1 == k2)
    {
        printf("\nPlease enter two different values of k.\n");
        return 1;
    }

    /*
     * First DSA signature
     */
    dsaSignature(messageHash, p, q, g,
                 privateKey, k1, &r1, &s1);

    /*
     * Second DSA signature
     */
    dsaSignature(messageHash, p, q, g,

```

```

        privateKey, k2, &r2, &s2);

printf("\n----- DSA SIGNATURE 1 -----\\n");
printf("k = %lld\\n", k1);
printf("r = %lld\\n", r1);
printf("s = %lld\\n", s1);

printf("\n----- DSA SIGNATURE 2 -----\\n");
printf("k = %lld\\n", k2);
printf("r = %lld\\n", r2);
printf("s = %lld\\n", s2);

printf("\n----- RESULT -----\\n");

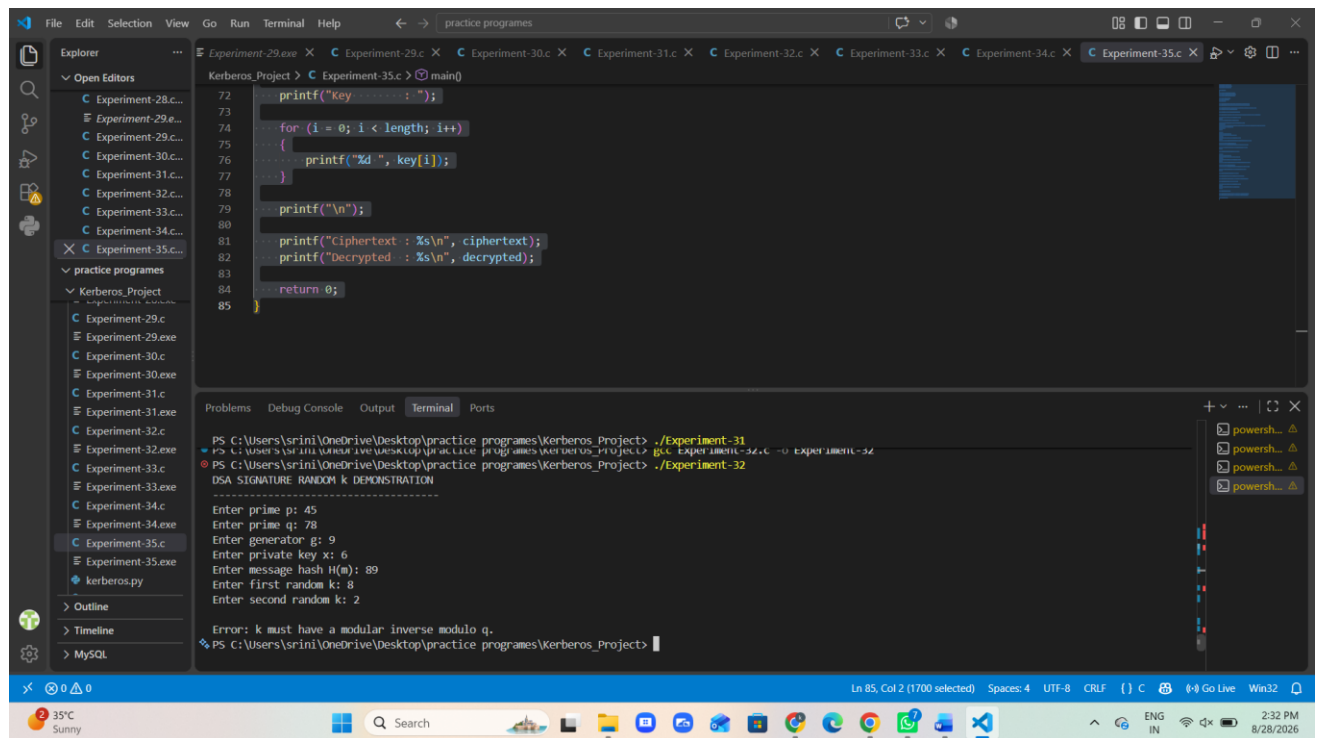
if (r1 != r2 || s1 != s2)
{
    printf("Same message was signed twice.\\n");
    printf("The DSA signatures are different.\\n");
    printf("This happens because a different random k was used.\\n");
}
else
{
    printf("The signatures are the same.\\n");
}

printf("\\nDSA implication:\\n");
printf("A fresh secret k makes signatures different for\\n");
printf("the same message on different occasions.\\n");

return 0;
}

```

OUTPUT:



33. Write a C program for Data encryption standard (DES) has been found vulnerable to very powerful attacks and therefore, the popularity of DES has been found slightly on the decline. DES is a block cipher and encrypts data in blocks of size of 64 bits each, which means 64 bits of plain text go as the input to DES, which produces 64 bits of ciphertext. The same algorithm and key are used for encryption and decryption, with minor differences. The key length is 56 bits. Implement in C programming.

PROGRAM:

```
#include <stdio.h>
#include <stdint.h>
```

```
/* Initial Permutation */
int IP[64] = {
    58,50,42,34,26,18,10,2,
    60,52,44,36,28,20,12,4,
    62,54,46,38,30,22,14,6,
    64,56,48,40,32,24,16,8,
    57,49,41,33,25,17,9,1,
    59,51,43,35,27,19,11,3,
    61,53,45,37,29,21,13,5,
    63,55,47,39,31,23,15,7
};
```

```
/* Final Permutation */
int FP[64] = {
    40,8,48,16,56,24,64,32,
    39,7,47,15,55,23,63,31,
    38,6,46,14,54,22,62,30,
    37,5,45,13,53,21,61,29,
    36,4,44,12,52,20,60,28,
    35,3,43,11,51,19,59,27,
    34,2,42,10,50,18,58,26,
    33,1,41,9,49,17,57,25
};
```

```
/* Expansion table */
```

```
int E[48] = {  
    32,1,2,3,4,5,  
    4,5,6,7,8,9,  
    8,9,10,11,12,13,  
    12,13,14,15,16,17,  
    16,17,18,19,20,21,  
    20,21,22,23,24,25,  
    24,25,26,27,28,29,  
    28,29,30,31,32,1  
};
```

```
/* Permutation after S-boxes */
```

```
int P[32] = {  
    16,7,20,21,  
    29,12,28,17,  
    1,15,23,26,  
    5,18,31,10,  
    2,8,24,14,  
    32,27,3,9,  
    19,13,30,6,  
    22,11,4,25  
};
```

```
/* Permuted Choice 1 */
```

```
int PC1[56] = {  
    57,49,41,33,25,17,9,  
    1,58,50,42,34,26,18,  
    10,2,59,51,43,35,27,  
    19,11,3,60,52,44,36,  
    63,55,47,39,31,23,15,  
    7,62,54,46,38,30,22,  
    14,6,61,53,45,37,29,  
    21,13,5,28,20,12,4  
};
```

```
/* Permuted Choice 2 */
```

```
int PC2[48] = {  
    14,17,11,24,1,5,  
    3,28,15,6,21,10,  
    23,19,12,4,26,8,  
    16,7,27,20,13,2,  
    41,52,31,37,47,55,  
    30,40,51,45,33,48,  
    44,49,39,56,34,53,  
    46,42,50,36,29,32  
};
```

```
/* Number of left shifts for each round */
```

```
int shifts[16] = {  
    1,1,2,2,2,2,2,2,  
    1,2,2,2,2,2,2,1  
};
```

```
/* DES S-boxes */
```

```
int S[8][4][16] = {
```

```
{
    {14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
    {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
    {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
    {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
},
```

```
{
    {15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
    {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
    {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
    {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
},
```

```
{
    {10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
    {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
    {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
    {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
},
```

```
{
    {7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
    {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
    {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
    {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}
},
```

```
{
    {2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
    {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
    {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
    {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
},
```

```
{
    {12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
    {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
    {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
    {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
},
```

```
{
    {4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
    {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
    {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
    {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
},
```

```
{
```

```

        {13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
        {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
        {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
        {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
    }
};

/* Apply permutation */
uint64_t permute(uint64_t input, int *table, int size)
{
    uint64_t output = 0;
    int i;

    for (i = 0; i < size; i++)
    {
        output <<= 1;
        output |= (input >> (64 - table[i])) & 1ULL;
    }

    return output;
}

/* Left rotate a 28-bit value */
uint32_t leftRotate28(uint32_t value, int count)
{
    return ((value << count) | (value >> (28 - count))) & 0xFFFFFFFF;
}

/* Generate 16 DES subkeys */
void generateKeys(uint64_t key, uint64_t subkeys[16])
{
    uint64_t permutedKey;
    uint32_t C, D;
    int i;

    permutedKey = permute(key, PC1, 56);

    C = (permutedKey >> 28) & 0xFFFFFFFF;
    D = permutedKey & 0xFFFFFFFF;

    for (i = 0; i < 16; i++)
    {
        C = leftRotate28(C, shifts[i]);
        D = leftRotate28(D, shifts[i]);

        uint64_t combined = ((uint64_t)C << 28) | D;

        subkeys[i] = 0;

        for (int j = 0; j < 48; j++)
        {
            subkeys[i] <<= 1;
            subkeys[i] |= (combined >> (56 - PC2[j])) & 1ULL;
        }
    }
}

```

```

    }
}

/* DES Feistel function */
uint32_t feistel(uint32_t R, uint64_t key)
{
    uint64_t expanded = 0;
    uint64_t xored;
    uint32_t output = 0;
    int i;

    /* Expansion 32 -> 48 */
    for (i = 0; i < 48; i++)
    {
        expanded <<= 1;
        expanded |= (R >> (32 - E[i])) & 1ULL;
    }

    /* XOR with round key */
    xored = expanded ^ key;

    /* S-box substitution */
    for (i = 0; i < 8; i++)
    {
        int sixBits = (xored >> (42 - 6 * i)) & 0x3F;

        int row = ((sixBits & 0x20) >> 4) |
            (sixBits & 0x01);

        int column = (sixBits >> 1) & 0x0F;

        int value = S[i][row][column];

        output = (output << 4) | value;
    }

    /* P permutation */
    uint32_t result = 0;

    for (i = 0; i < 32; i++)
    {
        result <<= 1;
        result |= (output >> (32 - P[i])) & 1U;
    }

    return result;
}

/* DES encryption/decryption */
uint64_t DES(uint64_t data, uint64_t subkeys[16], int decrypt)
{
    uint64_t permuted;
    uint32_t L, R;
    int i;

```

```

/* Initial permutation */
permuted = permute(data, IP, 64);

L = (uint32_t)(permuted >> 32);
R = (uint32_t)(permuted & 0xFFFFFFFF);

for (i = 0; i < 16; i++)
{
    uint32_t oldR = R;

    int keyIndex;

    if (decrypt)
        keyIndex = 15 - i;
    else
        keyIndex = i;

    R = L ^ feistel(R, subkeys[keyIndex]);
    L = oldR;
}

/* Swap L and R */
uint64_t combined = ((uint64_t)R << 32) | L;

/* Final permutation */
return permute(combined, FP, 64);
}

int main()
{
    uint64_t plaintext;
    uint64_t key;
    uint64_t ciphertext;
    uint64_t decrypted;
    uint64_t subkeys[16];

    printf("DATA ENCRYPTION STANDARD (DES)\n");
    printf("-----\n");

    printf("Enter 64-bit plaintext in hexadecimal: ");
    scanf("%llx", (unsigned long long *)&plaintext);

    printf("Enter 64-bit DES key in hexadecimal: ");
    scanf("%llx", (unsigned long long *)&key);

    /* Generate DES keys */
    generateKeys(key, subkeys);

    printf("\nGenerated 16 DES subkeys:\n");

    for (int i = 0; i < 16; i++)
    {
        printf("K%-2d = %012lX\n",

```

```

        i + 1,
        (unsigned long long)subkeys[i]);
    }

/* Encryption */
ciphertext = DES(plaintext, subkeys, 0);

printf("\nPlaintext = %016llX\n",
        (unsigned long long)plaintext);

printf("Ciphertext = %016llX\n",
        (unsigned long long)ciphertext);

/* Decryption */
decrypted = DES(ciphertext, subkeys, 1);

printf("Decrypted = %016llX\n",
        (unsigned long long)decrypted);

if (decrypted == plaintext)
{
    printf("\nDES encryption/decryption successful!\n");
}
else
{
    printf("\nDES verification failed!\n");
}

return 0;
}

```

OUTPUT:

```

PS C:\Users\sri\OneDrive\Desktop\practice programs\Kerberos_Project> ./Experiment-32
Error: k must have a modular inverse modulo q.
PS C:\Users\sri\OneDrive\Desktop\practice programs\Kerberos_Project> gcc Experiment-33.c -o Experiment-33
PS C:\Users\sri\OneDrive\Desktop\practice programs\Kerberos_Project> ./Experiment-33
DATA ENCRYPTION STANDARD (DES)
-----
Enter 64-bit plaintext in hexadecimal: r56jw3kab
Enter 64-bit DES key in hexadecimal:
Generated 16 DES subkeys:
K1 = FF34EBBA7F66
K2 = 598BFF5F5927
K3 = B5FCFB8669FC
K4 = F76FE6E18BD5
K5 = FAF95F3868B
K6 = 5DB77F5F1F0F
K7 = E7D4FF1E71FC
K8 = FFCBE661F9E5
K9 = F79F7B6EFA09
K10 = EFFAC7B3F55B
K11 = 3BFDEAF9722
K12 = 7C75BDC6F66
K13 = F7CD7D7CCADC
K14 = CFE8701F40B
K15 = 3FBFBFAFB629
K16 = E8DF8D3B568B

Plaintext = 0040203075398C1D
Ciphertext = B334C3B0A61E7E10
Decrypted = 0040203075398C1D

DES encryption/decryption successful
PS C:\Users\sri\OneDrive\Desktop\practice programs\Kerberos_Project>

```

34. Write a C program for ECB, CBC, and CFB modes, the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments). In other words, for these three modes, the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. One common method of padding, if needed, consists of a 1 bit followed by as few zero bits, possibly none, as are necessary to complete the final block. It is considered good practice for the sender to pad every message, including messages in which the final message block is already complete. What is the motivation for including a padding block when padding is not needed?

PROGRAM:

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define BLOCK_SIZE 8

/*
    Simple educational block cipher.
    This is NOT a secure cipher.
    It is used only to demonstrate ECB, CBC and CFB.
*/
uint64_t encryptBlock(uint64_t block, uint64_t key)
{
    return block ^ key;
}

/*
    Decrypt the simple XOR block cipher.
*/
uint64_t decryptBlock(uint64_t block, uint64_t key)
{
    return block ^ key;
}

/*
    Convert hexadecimal string to 64-bit integer.
*/
uint64_t hexToInt(char *str)
{
    uint64_t value = 0;
    sscanf(str, "%llx", (unsigned long long *)&value);
    return value;
}

/*
    Add 1-bit followed by 0-bits padding.
    For simplicity, each block is 64 bits.
*/
int addPadding(char *input, char *output)
{
    int len = strlen(input);
    int paddedLength;
    int i;

    /*
```

```

    Always add padding.
    If already a multiple of 8,
    add one complete padding block.
    */
    paddedLength = ((len / BLOCK_SIZE) + 1) * BLOCK_SIZE;

    for (i = 0; i < len; i++)
    {
        output[i] = input[i];
    }

    /*
        First padding byte represents the
        1-bit followed by zeros concept.
    */
    output[len] = 0x80;

    for (i = len + 1; i < paddedLength; i++)
    {
        output[i] = 0x00;
    }

    output[paddedLength] = '\0';

    return paddedLength;
}

/*
    ECB MODE
    */
void ECB_Encrypt(char *data, int length, uint64_t key)
{
    int i;

    printf("\nECB MODE\n");
    printf("-----\n");

    for (i = 0; i < length; i += BLOCK_SIZE)
    {
        uint64_t block = 0;
        uint64_t cipher;

        memcpy(&block, data + i, BLOCK_SIZE);

        cipher = encryptBlock(block, key);

        printf("Block %d: %016lX\n",
            (i / BLOCK_SIZE) + 1,
            (unsigned long long)cipher);
    }
}

/*
    CBC MODE

```

```

*/
void CBC_Encrypt(char *data,
                 int length,
                 uint64_t key,
                 uint64_t IV)
{
    int i;

    printf("\nCBC MODE\n");
    printf("-----\n");

    for (i = 0; i < length; i += BLOCK_SIZE)
    {
        uint64_t block = 0;
        uint64_t cipher;

        memcpy(&block, data + i, BLOCK_SIZE);

        /*
         CBC:
         C[i] = E(K, P[i] XOR C[i-1])
        */
        block = block ^ IV;

        cipher = encryptBlock(block, key);

        IV = cipher;

        printf("Block %d: %016lX\n",
              (i / BLOCK_SIZE) + 1,
              (unsigned long long)cipher);
    }
}

/*
 CFB MODE
*/
void CFB_Encrypt(char *data,
                 int length,
                 uint64_t key,
                 uint64_t IV)
{
    int i;

    printf("\nCFB MODE\n");
    printf("-----\n");

    for (i = 0; i < length; i += BLOCK_SIZE)
    {
        uint64_t plaintext = 0;
        uint64_t encryptedIV;
        uint64_t cipher;

        memcpy(&plaintext, data + i, BLOCK_SIZE);

```

```

/*
    CFB:
    C[i] = P[i] XOR E(K, IV)
*/
encryptedIV = encryptBlock(IV, key);

cipher = plaintext ^ encryptedIV;

IV = cipher;

printf("Block %d: %016lX\n",
       (i / BLOCK_SIZE) + 1,
       (unsigned long long)cipher);
}
}

int main()
{
    char input[256];
    char paddedData[264];

    int paddedLength;

    uint64_t key;
    uint64_t IV;

    printf("ECB, CBC AND CFB MODES\n");
    printf("-----\n");

    printf("Enter plaintext: ");
    scanf("%[^\n]", input);

    printf("Enter 64-bit key in hexadecimal: ");
    scanf("%llx", (unsigned long long *)&key);

    printf("Enter 64-bit IV in hexadecimal: ");
    scanf("%llx", (unsigned long long *)&IV);

    /*
    Apply padding
    */
    paddedLength = addPadding(input, paddedData);

    printf("\nOriginal plaintext length: %lu bytes",
           strlen(input));

    printf("\nPadded plaintext length: %d bytes\n",
           paddedLength);

    printf("\nPadding added using:\n");
    printf("1 bit followed by 0 bits\n");

    /*

```

```

        ECB
    */
    ECB_Encrypt(paddedData, paddedLength, key);

/*
    CBC
    */
    CBC_Encrypt(paddedData, paddedLength, key, IV);

/*
    CFB
    */
    CFB_Encrypt(paddedData, paddedLength, key, IV);

    return 0;
}

```

OUTPUT:

```

PS C:\Users\srin\OneDrive\Desktop\practice programs\Kerberos_Project> ./Experiment-33
DES encryption/decryption successful!
PS C:\Users\srin\OneDrive\Desktop\practice programs\Kerberos_Project> gcc Experiment-34.c -o Experiment-34
PS C:\Users\srin\OneDrive\Desktop\practice programs\Kerberos_Project> ./Experiment-34
ECB, CBC AND CFB MODES
-----
Enter plaintext: KANYA
Enter 64-bit key in hexadecimal: njdbu4h5665nij
Enter 64-bit IV in hexadecimal:
Original plaintext length: 5 bytes
Padded plaintext length: 8 bytes

Padding added using:
1 bit followed by 0 bits

ECB MODE
-----
Block 1: 00088041592F414B

CBC MODE
-----
Block 1: 00792C990E8C8294

CFB MODE
-----
Block 1: 00792C990E8C8294
PS C:\Users\srin\OneDrive\Desktop\practice programs\Kerberos_Project>

```

35. Write a C program for one-time pad version of the Vigenère cipher. In this scheme, the key is a

stream of random numbers between 0 and 26. For example, if the key is 3 19 5 . . . , then the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19 letters, the third with a shift of 5 letters, and so on

PROGRAM:

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

```

```

int main()
{
    char plaintext[100];

```

```

char ciphertext[100];
char decrypted[100];

int key[100];
int length;
int i;

printf("ONE-TIME PAD VIGENERE CIPHER\n");
printf("-----\n");

printf("Enter plaintext (CAPITAL letters only): ");
scanf("%s", plaintext);

length = strlen(plaintext);

printf("\nEnter %d random key values (0-25):\n", length);

for (i = 0; i < length; i++)
{
    scanf("%d", &key[i]);

    if (key[i] < 0 || key[i] > 25)
    {
        printf("Invalid key! Enter values between 0 and 25.\n");
        return 1;
    }
}

/*
    Encryption
    C = (P + K) mod 26
*/
for (i = 0; i < length; i++)
{
    int plaintextValue = plaintext[i] - 'A';

    int cipherValue =
        (plaintextValue + key[i]) % 26;

    ciphertext[i] = cipherValue + 'A';
}

ciphertext[length] = '\0';

/*
    Decryption
    P = (C - K + 26) mod 26
*/
for (i = 0; i < length; i++)
{
    int cipherValue = ciphertext[i] - 'A';

    int plainValue =
        (cipherValue - key[i] + 26) % 26;

```

```

        decrypted[i] = plainValue + 'A';
    }

    decrypted[length] = '\0';

    printf("\n----- RESULT ----- \n");

    printf("Plaintext : %s\n", plaintext);

    printf("Key      : ");

    for (i = 0; i < length; i++)
    {
        printf("%d ", key[i]);
    }

    printf("\n");

    printf("Ciphertext : %s\n", ciphertext);
    printf("Decrypted  : %s\n", decrypted);

    return 0;
}

```

OUTPUT:

```

PS C:\Users\sri\OneDrive\Desktop\practice programmes\Kerberos_Project> ./Experiment-34
Block 1: 00792C90E08C2904
PS C:\Users\sri\OneDrive\Desktop\practice programmes\Kerberos_Project> gcc Experiment-35.c -o Experiment-35
PS C:\Users\sri\OneDrive\Desktop\practice programmes\Kerberos_Project> ./Experiment-35
ONE-TIME PAD VIGENERE CIPHER
-----
Enter plaintext (CAPITAL letters only): KAVYA
Enter 5 random key values (0-25):
4 5 7 2 0
----- RESULT -----
Plaintext : KAVYA
Key       : 4 5 7 2 0
Ciphertext: OFCAA
Decrypted : KAVYA
PS C:\Users\sri\OneDrive\Desktop\practice programmes\Kerberos_Project>

```

36. Write a C program for Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$

and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int mod_inverse(int a)
{
    int i;

    for (i = 0; i < 26; i++)
    {
        if ((a * i) % 26 == 1)
            return i;
    }

    return -1;
}

int main()
{
    char plaintext[200];
    char ciphertext[200];
    char decrypted[200];

    int a, b;
    int i, p, c;
    int inverse_a;

    printf("AFFINE CAESAR CIPHER\n");

    printf("Enter plaintext: ");
    fgets(plaintext, sizeof(plaintext), stdin);

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);

    a = a % 26;
    b = b % 26;
```

```

if (gcd(a, 26) != 1)
{
    printf("\nInvalid value of a!\n");
    printf("a must be relatively prime to 26.\n");
    printf("Valid values: 1,3,5,7,9,11,15,17,19,21,23,25\n");
    return 0;
}

inverse_a = mod_inverse(a);

/* Encryption */
for (i = 0; plaintext[i] != '\0'; i++)
{
    if (isalpha(plaintext[i]))
    {
        p = toupper(plaintext[i]) - 'A';
        c = (a * p + b) % 26;
        ciphertext[i] = c + 'A';
    }
    else
    {
        ciphertext[i] = plaintext[i];
    }
}

ciphertext[i] = '\0';

/* Decryption */
for (i = 0; ciphertext[i] != '\0'; i++)
{
    if (isalpha(ciphertext[i]))
    {
        c = ciphertext[i] - 'A';
        p = (inverse_a * (c - b + 26)) % 26;
        decrypted[i] = p + 'A';
    }
    else
    {
        decrypted[i] = ciphertext[i];
    }
}

decrypted[i] = '\0';

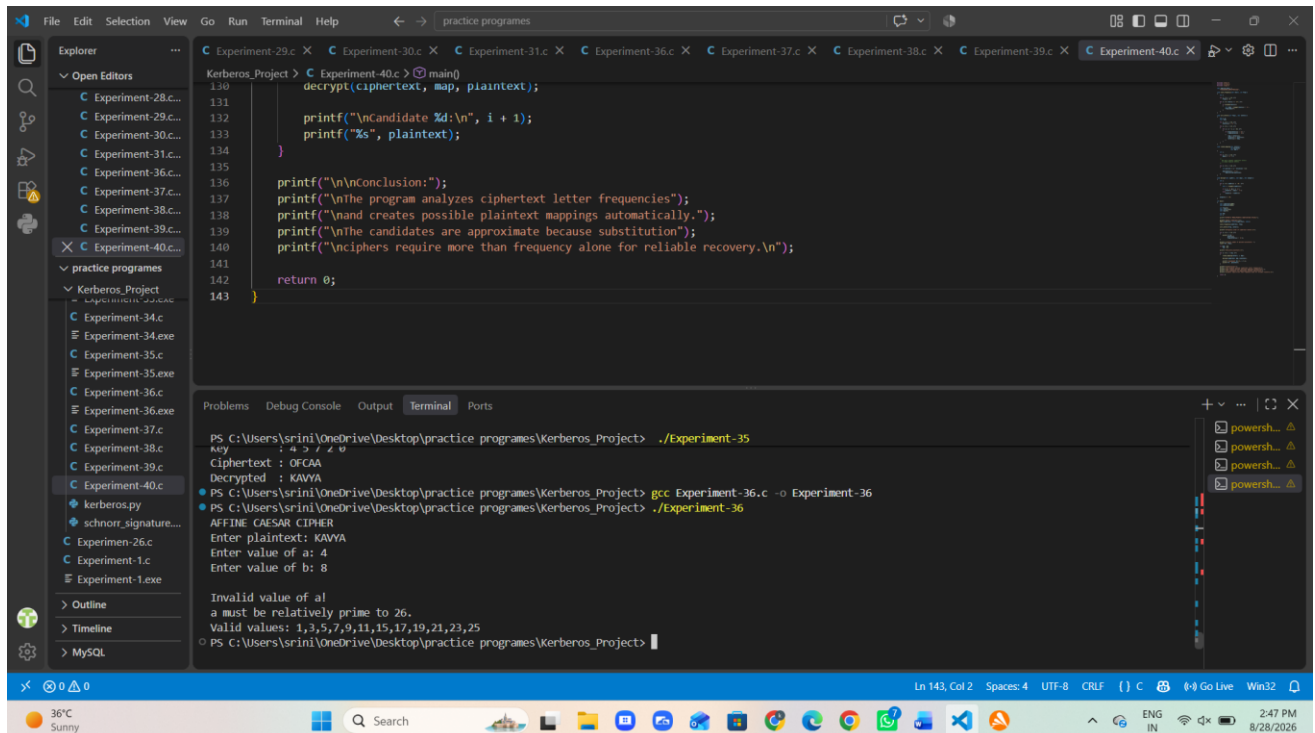
printf("\nCiphertext: %s", ciphertext);
printf("Decrypted text: %s", decrypted);

printf("\n\nFormula:");
printf("\nEncryption:  $C = (aP + b) \bmod 26$ ");
printf("\nDecryption:  $P = a^{-1}(C - b) \bmod 26$ \n");

return 0;
}

```

OUTPUT:



```
File Edit Selection View Go Run Terminal Help
practice_programes

Explorer
Open Editors
Experiment-28.c
Experiment-29.c
Experiment-30.c
Experiment-31.c
Experiment-36.c
Experiment-37.c
Experiment-38.c
Experiment-39.c
Experiment-40.c
practice_programes
Kerberos_Project
Experiment-34.c
Experiment-34.exe
Experiment-35.c
Experiment-35.exe
Experiment-36.c
Experiment-36.exe
Experiment-37.c
Experiment-38.c
Experiment-39.c
Experiment-40.c
kerberos.py
schnorr_signature...
Experiment-26.c
Experiment-1.c
Experiment-1.exe
Outline
Timeline
MySQL

Experiment-40.c
130 decrypt(ciphertext, map, plaintext);
131
132 printf("\nCandidate %d:\n", i + 1);
133 printf("%s", plaintext);
134 }
135
136 printf("\n\nConclusion:");
137 printf("\n\nThe program analyzes ciphertext letter frequencies");
138 printf("\n\nand creates possible plaintext mappings automatically.");
139 printf("\n\nThe candidates are approximate because substitution");
140 printf("\n\nnciphers require more than frequency alone for reliable recovery.\n\n");
141
142 return 0;
143 }

Problems Debug Console Output Terminal Ports
PS C:\Users\srin\OneDrive\Desktop\practice_programes\Kerberos_Project> ./Experiment-35
Key : 4 7 7 4 0
Ciphertext : QFCAA
Decrypted : KAVYA
PS C:\Users\srin\OneDrive\Desktop\practice_programes\Kerberos_Project> gcc Experiment-36.c -o Experiment-36
PS C:\Users\srin\OneDrive\Desktop\practice_programes\Kerberos_Project> ./Experiment-36
AFFINE CAESAR CIPHER
Enter plaintext: KAVYA
Enter value of a: 4
Enter value of b: 8

Invalid value of a
a must be relatively prime to 26.
Valid values: 1,3,5,7,9,11,15,17,19,21,23,25
PS C:\Users\srin\OneDrive\Desktop\practice_programes\Kerberos_Project>
```

37. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
typedef struct
```

```
{
    char letter;
    int count;
} Frequency;
```

```
void sort_frequency(Frequency f[])
```

```
{
    int i, j;
    Frequency temp;

    for (i = 0; i < 26; i++)
    {
        for (j = i + 1; j < 26; j++)
        {
            if (f[j].count > f[i].count)
            {
                temp = f[i];
                f[i] = f[j];
                f[j] = temp;
            }
        }
    }
}
```

```
}  
}
```

```
void decrypt_text(char cipher[], char map[], int shift, char output[])
```

```
{  
    int i;  
    char c;  
  
    for (i = 0; cipher[i] != '\0'; i++)  
    {  
        c = toupper(cipher[i]);  
  
        if (c >= 'A' && c <= 'Z')  
        {  
            int index = c - 'A';  
            int mapped = map[index] - 'A';  
  
            mapped = (mapped + shift) % 26;  
            output[i] = mapped + 'A';  
        }  
        else  
        {  
            output[i] = cipher[i];  
        }  
    }  
  
    output[i] = '\0';  
}
```

```
int main()
```

```
{  
    char ciphertext[1000];  
    char plaintext[1000];  
  
    Frequency freq[26];  
  
    char english[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";  
  
    int i, j;  
    int top;  
  
    printf("MONOALPHABETIC SUBSTITUTION FREQUENCY ATTACK\n");  
  
    printf("\nEnter ciphertext:\n");  
    fgets(ciphertext, sizeof(ciphertext), stdin);  
  
    for (i = 0; i < 26; i++)  
    {  
        freq[i].letter = 'A' + i;  
        freq[i].count = 0;  
    }  
  
    for (i = 0; ciphertext[i] != '\0'; i++)  
    {
```

```

    if (isalpha(ciphertext[i]))
    {
        int index = toupper(ciphertext[i]) - 'A';
        freq[index].count++;
    }
}

sort_frequency(freq);

printf("\nLetter frequencies:\n");

for (i = 0; i < 26; i++)
{
    printf("%c : %d\n",
           freq[i].letter,
           freq[i].count);
}

printf("\nEnter number of possible plaintexts to display: ");
scanf("%d", &top);

if (top > 26)
    top = 26;

printf("\nPossible plaintexts:\n");

for (j = 0; j < top; j++)
{
    char map[26];

    for (i = 0; i < 26; i++)
        map[i] = 'A' + i;

    for (i = 0; i < 26; i++)
    {
        if (freq[i].count > 0)
        {
            int english_index = (i + j) % 26;
            map[freq[i].letter - 'A'] =
                english[english_index];
        }
    }

    decrypt_text(ciphertext, map, 0, plaintext);

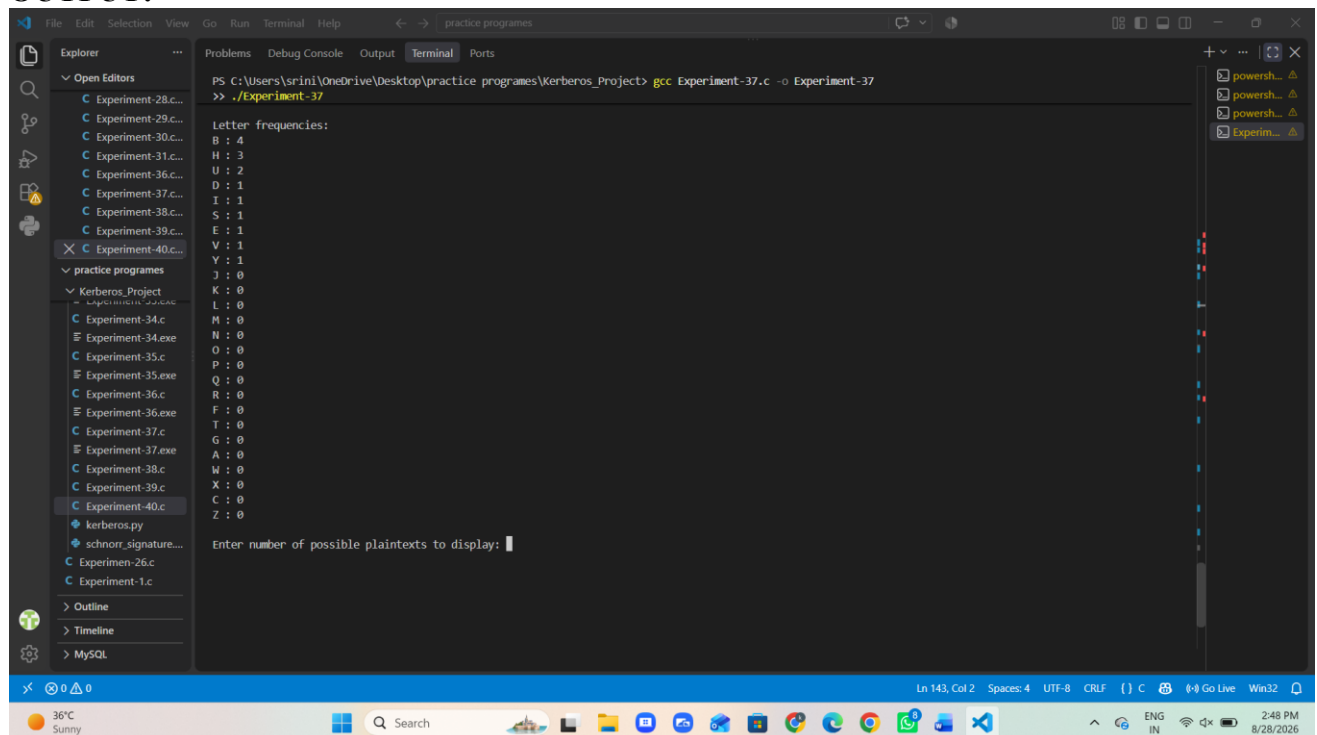
    printf("\nCandidate %d:\n%s", j + 1, plaintext);
}

printf("\n\nNote:");
printf("\nThis is a frequency-analysis heuristic.");
printf("\nFor a completely automatic high-quality attack,");
printf("\nword dictionaries and n-gram scoring are normally used.\n");

return 0;

```

}
OUTPUT:



```
PS C:\Users\srin\OneDrive\Desktop\practice_programes\Kerberos_Project> gcc Experiment-37.c -o Experiment-37
>> ./Experiment-37

Letter frequencies:
B : 4
H : 3
U : 2
D : 1
I : 1
S : 1
E : 1
V : 1
Y : 1
J : 0
K : 0
L : 0
M : 0
N : 0
O : 0
P : 0
Q : 0
R : 0
F : 0
T : 0
G : 0
A : 0
W : 0
X : 0
C : 0
Z : 0

Enter number of possible plaintexts to display: 
```

38. Write a C program for Hill cipher succumbs to a known plaintext attack if sufficient plaintext– ciphertext pairs are provided. It is even easier to solve the Hill cipher if a chosen plaintext attack can be mounted. Implement in C programming.

PROGRAM:

```
#include <stdio.h>
```

```
int mod26(int x)
```

```
{
    x = x % 26;
```

```
    if (x < 0)
        x += 26;
```

```
    return x;
```

```
}
```

```
int inverse_mod26(int x)
```

```
{
    int i;
```

```
    for (i = 1; i < 26; i++)
    {
        if ((x * i) % 26 == 1)
            return i;
    }
```

```
    return -1;
```

```
}
```

```
int main()
```

```

{
    int P[2][2];
    int C[2][2];

    int K[2][2];

    int det;
    int inv_det;

    int i, j;

    printf("HILL CIPHER KNOWN PLAINTEXT ATTACK\n");

    printf("\nEnter two plaintext blocks as numbers.");
    printf("\nUse A=0, B=1, ..., Z=25\n");

    printf("\nPlaintext block 1:\n");
    scanf("%d %d", &P[0][0], &P[1][0]);

    printf("Plaintext block 2:\n");
    scanf("%d %d", &P[0][1], &P[1][1]);

    printf("\nEnter corresponding ciphertext blocks:\n");

    printf("Ciphertext block 1:\n");
    scanf("%d %d", &C[0][0], &C[1][0]);

    printf("Ciphertext block 2:\n");
    scanf("%d %d", &C[0][1], &C[1][1]);

    /*
    
$$K = C * \text{inverse}(P) \bmod 26$$


    
$$P = \begin{bmatrix} p1 & p2 \\ p3 & p4 \end{bmatrix}$$


    
$$\text{inverse}(P) = \frac{1}{\text{determinant}} \begin{bmatrix} p4 & -p2 \\ -p3 & p1 \end{bmatrix}$$

    */

    det = mod26(P[0][0] * P[1][1]
        - P[0][1] * P[1][0]);

    inv_det = inverse_mod26(det);

    if (inv_det == -1)
    {
        printf("\nKey cannot be recovered.");
        printf("\nPlaintext matrix is not invertible modulo 26.\n");
        return 0;
    }
}

```

```

/* K = C * inverse(P) */

K[0][0] = mod26(
    inv_det *
    (C[0][0] * P[1][1]
     - C[0][1] * P[1][0])
);

K[0][1] = mod26(
    inv_det *
    (-C[0][0] * P[0][1]
     + C[0][1] * P[0][0])
);

K[1][0] = mod26(
    inv_det *
    (C[1][0] * P[1][1]
     - C[1][1] * P[1][0])
);

K[1][1] = mod26(
    inv_det *
    (-C[1][0] * P[0][1]
     + C[1][1] * P[0][0])
);

printf("\nRecovered Hill Cipher Key Matrix:\n");

for (i = 0; i < 2; i++)
{
    for (j = 0; j < 2; j++)
        printf("%d ", K[i][j]);

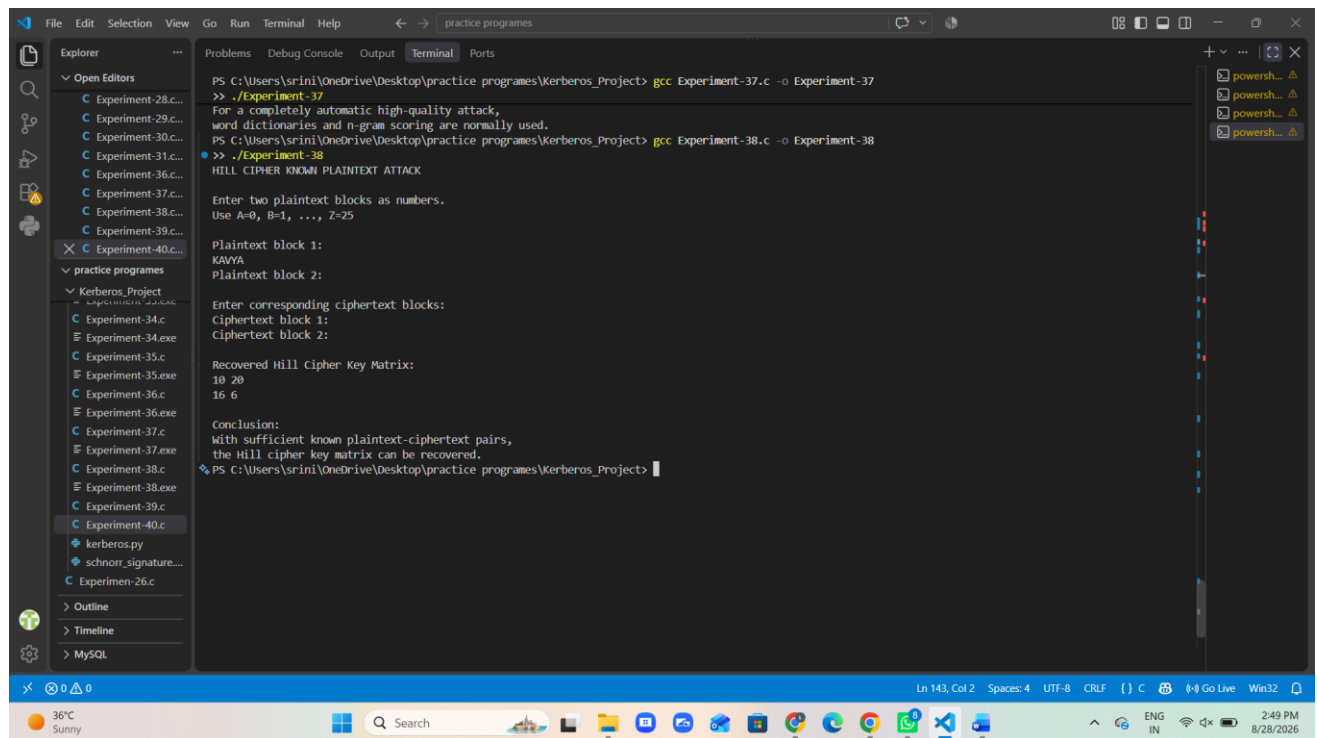
    printf("\n");
}

printf("\nConclusion:");
printf("\nWith sufficient known plaintext-ciphertext pairs,");
printf("\nthe Hill cipher key matrix can be recovered.\n");

return 0;
}

```

OUTPUT:



39. Write a C program that can perform a letter frequency attack on an additive cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
double english_freq[26] =
{
    8.2, 1.5, 2.8, 4.3, 12.7, 2.2,
    2.0, 6.1, 7.0, 0.15, 0.77, 4.0,
    2.4, 6.7, 7.5, 1.9, 0.095, 6.0,
    6.3, 9.1, 2.8, 0.98, 2.4, 0.15,
    2.0, 0.074
};

double calculate_score(char text[])
{
    int count[26] = {0};
    int total = 0;
    int i;
    double score = 0.0;

    for (i = 0; text[i] != '\0'; i++)
    {
        if (isalpha(text[i]))
        {
            int index = toupper(text[i]) - 'A';
            count[index]++;
            total++;
        }
    }
}
```

```

    if (total == 0)
        return 999999;

    for (i = 0; i < 26; i++)
    {
        double actual =
            ((double)count[i] / total) * 100;

        score += (actual - english_freq[i])
            * (actual - english_freq[i]);
    }

    return score;
}

void decrypt(char cipher[], int key, char output[])
{
    int i;

    for (i = 0; cipher[i] != '\0'; i++)
    {
        if (isalpha(cipher[i]))
        {
            int c = toupper(cipher[i]) - 'A';
            int p = (c - key + 26) % 26;

            output[i] = p + 'A';
        }
        else
        {
            output[i] = cipher[i];
        }
    }

    output[i] = '\0';
}

int main()
{
    char ciphertext[1000];
    char plaintext[1000];

    int key;
    int top;

    double score;

    printf("ADDITIVE CIPHER FREQUENCY ATTACK\n");

    printf("\nEnter ciphertext:\n");
    fgets(ciphertext, sizeof(ciphertext), stdin);

    printf("\nEnter number of possible plaintexts to display: ");

```

```

scanf("%d", &top);

if (top > 26)
    top = 26;

printf("\nTrying all 26 possible keys...\n");

for (key = 0; key < top; key++)
{
    decrypt(ciphertext, key, plaintext);

    score = calculate_score(plaintext);

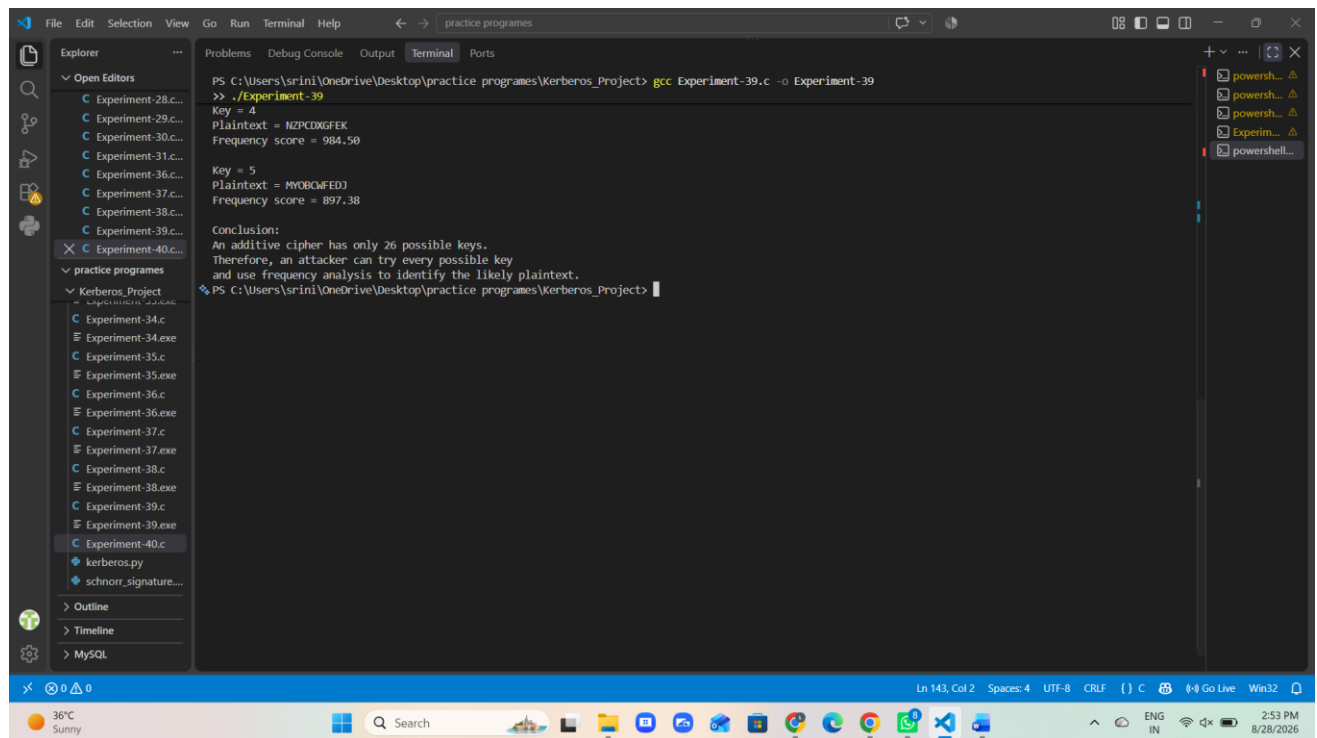
    printf("\nKey = %d", key);
    printf("\nPlaintext = %s", plaintext);
    printf("Frequency score = %.2f\n", score);
}

printf("\nConclusion:");
printf("\nAn additive cipher has only 26 possible keys.");
printf("\nTherefore, an attacker can try every possible key");
printf("\nand use frequency analysis to identify the likely plaintext.\n");

return 0;
}

```

OUTPUT:



40. Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
char english_order[] =
    "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
```

```
void count_frequency(char text[], int freq[])
{
    int i;

    for (i = 0; i < 26; i++)
        freq[i] = 0;

    for (i = 0; text[i] != '\0'; i++)
    {
        if (isalpha(text[i]))
        {
            int index = toupper(text[i]) - 'A';
            freq[index]++;
        }
    }
}
```

```
void sort_letters(int freq[], char letters[])
{

```

```

int i, j;
char temp;

for (i = 0; i < 26; i++)
    letters[i] = 'A' + i;

for (i = 0; i < 25; i++)
{
    for (j = i + 1; j < 26; j++)
    {
        if (freq[letters[j] - 'A'] >
            freq[letters[i] - 'A'])
        {
            temp = letters[i];
            letters[i] = letters[j];
            letters[j] = temp;
        }
    }
}

}

void create_mapping(char letters[],
                    int variation,
                    char map[])
{
    int i;

    for (i = 0; i < 26; i++)
        map[i] = 'A' + i;

    /*
     * Map most frequent ciphertext letters
     * to common English letters.
     */

    for (i = 0; i < 26; i++)
    {
        int position = (i + variation) % 26;

        map[letters[i] - 'A'] =
            english_order[position];
    }
}

void decrypt(char cipher[], char map[], char output[])
{
    int i;

    for (i = 0; cipher[i] != '\0'; i++)
    {
        char c = toupper(cipher[i]);

        if (c >= 'A' && c <= 'Z')
            output[i] = map[c - 'A'];
    }
}

```

```

        else
            output[i] = cipher[i];
    }

    output[i] = '\0';
}

int main()
{
    char ciphertext[2000];
    char plaintext[2000];

    int freq[26];
    char letters[26];
    char map[26];

    int top;
    int i;

    printf("AUTOMATIC MONOALPHABETIC SUBSTITUTION ATTACK\n");

    printf("\nEnter ciphertext:\n");
    fgets(ciphertext, sizeof(ciphertext), stdin);

    count_frequency(ciphertext, freq);

    sort_letters(freq, letters);

    printf("\nFrequency order of ciphertext letters:\n");

    for (i = 0; i < 26; i++)
    {
        printf("%c(%d) ",
            letters[i],
            freq[letters[i] - 'A']);
    }

    printf("\n\nEnter number of possible plaintexts: ");
    scanf("%d", &top);

    if (top > 26)
        top = 26;

    printf("\nPossible plaintexts:\n");

    for (i = 0; i < top; i++)
    {
        create_mapping(letters, i, map);

        decrypt(ciphertext, map, plaintext);

        printf("\nCandidate %d:\n", i + 1);
        printf("%s", plaintext);
    }
}

```

```

printf("\n\nConclusion:");
printf("\nThe program analyzes ciphertext letter frequencies");
printf("\nand creates possible plaintext mappings automatically.");
printf("\nThe candidates are approximate because substitution");
printf("\nciphers require more than frequency alone for reliable recovery.\n");

return 0;
}
OUTPUT:

```

```

PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> gcc Experiment-39.c -o Experiment-39
>> ./Experiment-39
An additive cipher has only 26 possible keys.
Therefore, an attacker can try every possible key
and use frequency analysis to identify the likely plaintext.
PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> gcc Experiment-40.c -o Experiment-40
>> ./Experiment-40
AUTOMATIC MONOALPHABETIC SUBSTITUTION ATTACK

Enter ciphertext:
budhifhudi

Frequency order of ciphertext letters:
D(2) H(2) I(2) U(2) F(1) B(1) G(0) A(0) E(0) J(0) K(0) L(0) M(0) N(0) O(0) P(0) Q(0) R(0) S(0) T(0) C(0) V(0) W(0) X(0) Y(0) Z(0)

Enter number of possible plaintexts: 67

Possible plaintexts:

Candidate 1:
NOETAITOE A

Candidate 2:
SITAOHATIO

Candidate 3:
HWAQISQWAI

Candidate 4:
RSQDHHISON

Candidate 5:
DHNSRHHIS

Candidate 6:
LRNSHDSRHH

Candidate 7:
CDSHRLHDSR

```

```

PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project> gcc Experiment-40.c -o Experiment-40
>> ./Experiment-40

Candidate 19:
XKPBVBIBKPV

Candidate 20:
QIBVQXVIBK

Candidate 21:
ZXVCJQXVJ

Candidate 22:
EQKXZJQX

Candidate 23:
TZJXQXZJQ

Candidate 24:
ADQZTQDXZ

Candidate 25:
OTQZEAZTQE

Candidate 26:
IAZETOEAZT

Conclusion:
The program analyzes ciphertext letter frequencies
and creates possible plaintext mappings automatically.
The candidates are approximate because substitution
ciphers require more than frequency alone for reliable recovery.
PS C:\Users\srin\OneDrive\Desktop\practice programmes\Kerberos_Project>

```